课堂笔记 · 16 节 · 40 幅配图

第 05 课 | 软件工程的来龙去脉：从需求鸿沟到可检查的承诺

回看原视频

沿用上一版已记录的转写阅读：20 个分块、491 条来源单元，覆盖 00:00.83 – 01:39:59.18。本次只调整第五课的图文组织与自测交互；未重新转写或逐秒听审。

课堂正文依据转写；2026-10-02 冻结的官方网页讲义共 39 页已逐页读取及目视，页面可能包含课后补注。图内紫色 AIGC 标识属于原讲义，不能自动视作讲师当堂口述。

讲师在约 01:39:50 请学生课后阅读剩余 slides。第 31 – 39 页现按知识点插入相关章节，均标“讲义延伸 · 课堂未展开”，不为其虚构课堂时间。引用只回查本地冻结讲义，未完整阅读其引用的原始论文或 796 页 UML 规范。

官方页图来自完整网页截图，并按原页序冻结为 source-slides.pdf（截图汇编，非官方原生 PDF）；讲义图的时间仅用于关联讲解。第 7 页的视频占位未作为成品图，演示改用 21:14.79 的原视频帧。

原视频为 1280×410 的课堂全景，演示区域约 720 像素宽；保留的设备与板书帧用于展示实际状态，不能恢复源视频本来没有的细节。

新闻、产业预测、学习成本与模型能力评价保留为讲师当时的判断，没有当作今天已独立核实的产品事实。本次桌面、390px 窄屏与打印检查另记于修订

review；未做真实手机实测。

先读摘要

本课用一个贯穿问题连接历史与 AI：怎样在时间、成本、团队和真实环境约束下，把人的意图变成能运行、能维护的系统？

代码生成变便宜以后，接口抽象、先做原型、尽早得到反馈、明确责任和保持需求可追踪仍然重要。Office Assistant 的设备实验展示了这些原则怎样与 agent 合作。

Royce 的经典文章强调顺序推进的风险；数据依赖图解释了晚发现错误为何造成大返工，也解释了 MVP、TDD 与敏捷怎样缩短反馈。

契约与 UML 把需要跨实现保持的决定保存下来。官方第 31 – 39 页的图与解读已融入相应知识点：默认决定、契约与实现、图与模型、语义边界、可检查约束及有意的留白；来源在就近位置标明。

00:00:00—00:09:16

开场：实时决策模型与 AI 加速

课程从近期 AI 进展切入。教师播放 TypeSafe AI 的 Jev 实时玩 Doom 的演示，讨论把游戏状态变成结构化输入、快速给出决策的交互。他猜测模型可能类似 prefill-only Transformer、以动作输出代替长段自然语言；这是现场推测，不能写成已经确认的模型架构。

这个能力的意义不只在游戏。若模型能够低延迟感知对话和环境，它就能驱动数字人的微表情、机器人反馈和陪伴场景，让交互从“等一句完整回答”变成连续的社会性反

应。教师把这视为一个可能被迅速打开的新生态。

教师把“智能”与“可用的交互延迟”分开：过去模型即使有理解能力，若需要等待一整段回答，也难以实时产生表情、动作或自然的对话反馈。微表情、数字人和机器人是他据此提出的应用设想，尚非本课完成的工程验证。

> **整理者说明**
>
> 官方讲义第 1 页另有技术介绍链接及并行输出的补注；本笔记没有核验该模型实现。AI 历史视频中的个别里程碑与调侃没有作为独立事实引用。



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 1 · 原始讲义 · 相关讲解 00:00:00（不是同秒视频截图） · 原讲义第 1/39 页：Review & Comments。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

随后课程回顾从早期 AI、搜索算法、AlexNet、Transformer、GPT-3、多模态到推理模型的发展，强调技术演进已经明显加速。生成视频、开放素材库与模型能力结合

后，内容生产成本也会急剧下降。

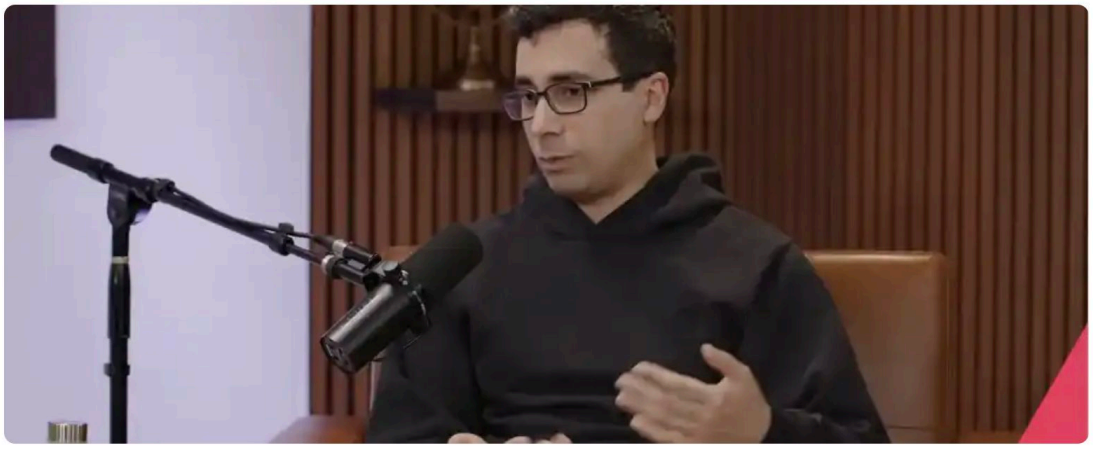
教师提醒，模型当前仍容易追逐立即奖励：它会讨好 prompt、用最快最脏的方式完成任务，缺少长期品位。但这种缺陷未必永久存在，因为“接下来做什么”的判断也可能通过更长反馈、合成数据和强化学习被训练。

Taste 在本课中被具体定义为“对接下来该做什么的直觉”。模型受即时奖励驱动，可能把任务快速做完，却留下长期结构问题。课堂引用 Noam Brown 的判断：当 AI 做掉大部分工作，人的注意力会转向它暂时不会的部分，但不能据此认定剩余部分永久不可训练。

工作性质在改变

OpenAI Noam Brown 的访谈

- 如果 AI 能做一个人 **90%** 的工作，那个人的注意力就会集中到 AI 做不好的 **10%** 上
- Taste (“**对接下来该做什么**” 的直觉)：他预计，一两代模型之后就比他强了
  - “可验证” v.s. “不可验证”



2 / 39

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 2 · 原始讲义 · 相关讲解 00:00:00（不是同秒视频截图） · 原讲义第 2/39 页：工作性质在改变。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

> 案例回扣 低延迟模型根据对话实时生成数字人表情。

> 案例回扣 Doom 演示用于提出“决策是否一定要逐字生成”的问题；毫秒级反馈可服务实时交互。

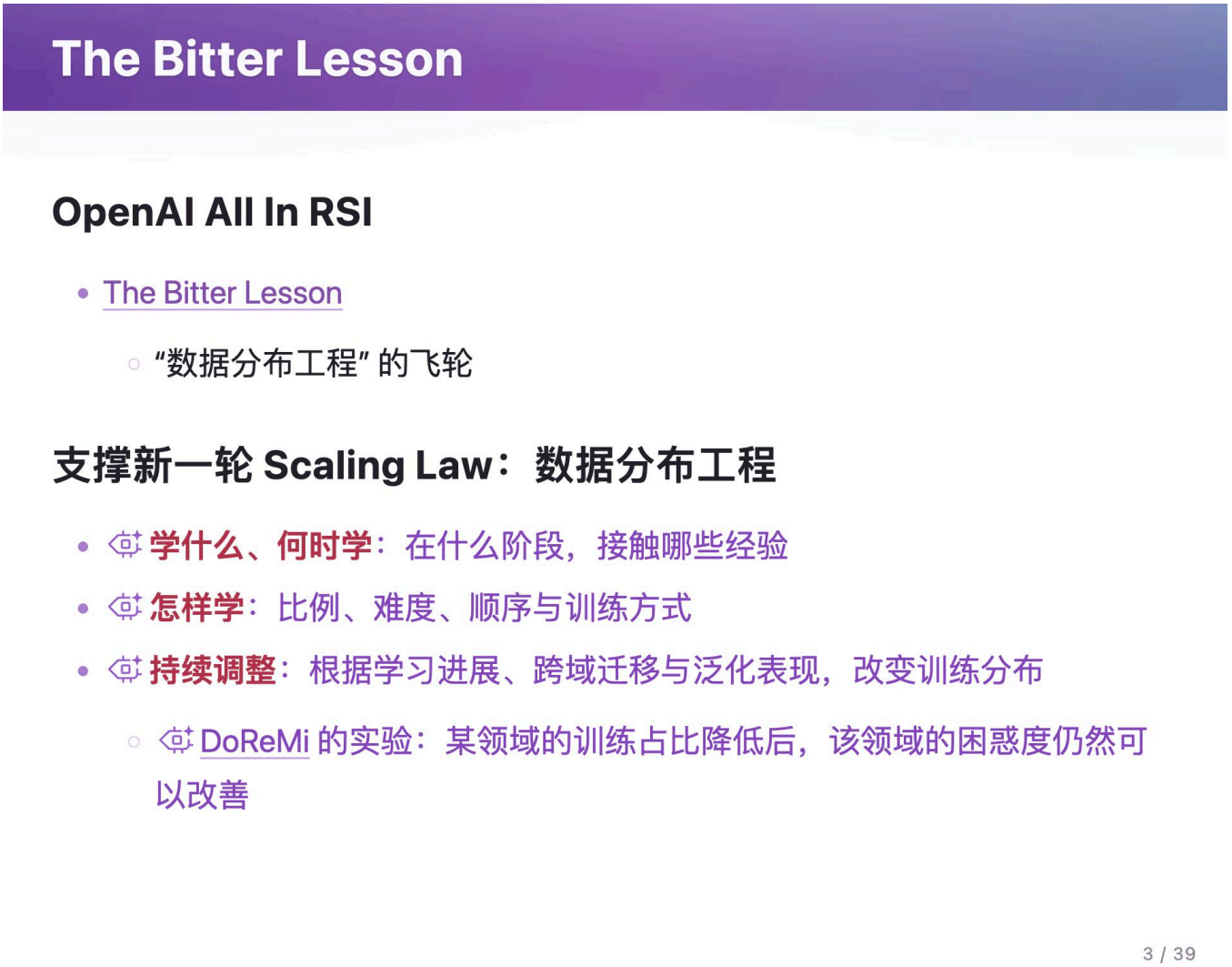
00:09:16—00:18:34

Scaling Law 的新阶段：数据分布工程

“可验证/不可验证”不是一条简单的边界。写数学证明虽然有形式化工具，仍须检查自然语言问题是否被正确翻译：如果把原问题形式化成一个空洞命题，证明器证明成功也没有解决原问题。反过来，研究品位、系统架构等难评价任务，也许能通过更晚、更昂贵的后续结果得到反馈。

教师提出一个推测：基础模型公司正在从“单纯扩大语料和参数”走向“数据分布工程”。模型先在语言上形成通用基座，再补入数学、多模态、空间推理、生物实验等领域的数据与反馈；每加入一个新领域，已有能力可能降低新领域所需的数据量，并反过来改善其他任务。

讲义补充把“数据分布工程”拆为三个可操作维度：学什么与何时学；各类数据的比例、难度、顺序及训练方式；依据学习进展、跨域迁移与泛化表现持续调整。飞轮需要独立验证的净收益，不能用模型自评分数升高代替真实能力提升。



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 3 · 原始讲义 · 相关讲解 00:09:16（不是同秒视频截图） · 原讲义第 3/39 页：The Bitter Lesson。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

他将这一过程描述成飞轮：选择新领域，建设可验证反馈，优化训练数据比例，再把新能力并回基座。只要训练稳定、训推一致且资源充足，规模与反馈闭环可能比局部算法改进更有决定性。

数据分布工程的飞轮

自动化能力生产过程

- 增强后的模型参与寻找领域、构建环境与验证器，并优化数据分布与训练方法。



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 4 · 原始讲义 · 相关讲解 00:09:16（不是同秒视频截图） · 原讲义第 4/39 页：数据分布工程的飞轮。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

教师用模拟器训练作反例：从零训练的模型可能利用画面边角像素、光照等偶然线索获取奖励，换一个环境就失败。已有语言、数学和代码能力的基座，能否以较少空间数据获得可迁移能力，是他的解释假说；“每加一个领域都提升全部任务”不是本课证明的定律。

人的学习常从与世界互动开始，再到语言、数学和研究判断；大模型的发展路径不同。教师用这种次序对比说明通用能力可能相互支撑，不应把它读成严格的认知科学实验结论。

这一判断也带来研究生态焦虑。小规模学术实验很难看到大厂真实的数据分布；一篇 blog 的新想法可能直接被基础模型团队快速验证。教师认为学术界需要重新思考还能

贡献什么，以及哪些“品位、系统架构、实验领域”暂时仍值得深入。

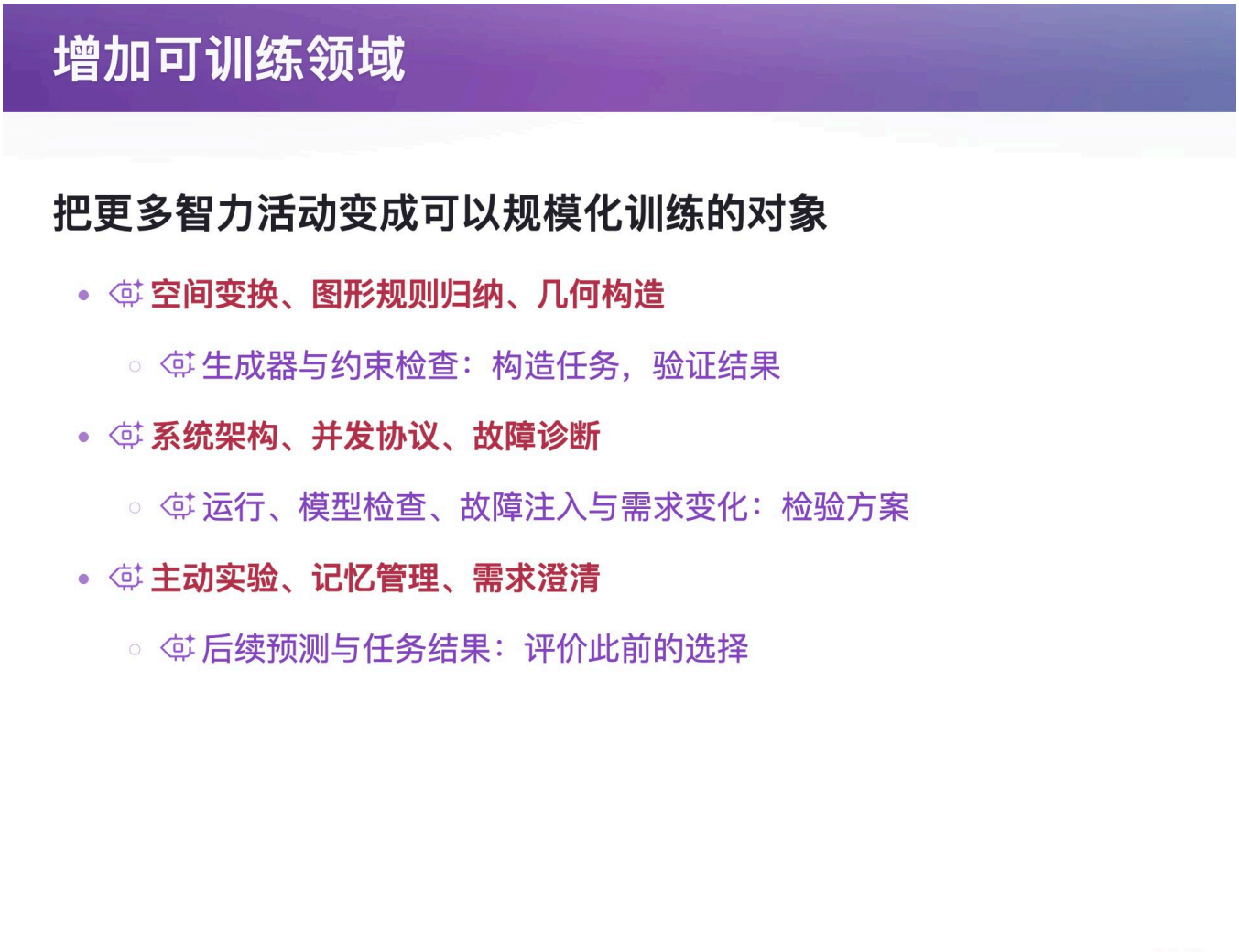
课程没有把焦虑当成放弃学习的理由。教师用“斩杀线不断上移”形容竞争环境：人仍需理解和驾驭 AI，努力成为能定义问题、判断结果并建立长期意义的人。

> **整理者说明**
> “数据分布工程”“飞轮”是教师对基础模型路线的分析框架，而非公开披露的厂商训练配方。

> 案例回扣 语言→数学→多模态→空间推理，被描述为能力逐层注入的路径。

> 案例回扣 新建实验室并建立可验证反馈，被类比为给基座模型安装一个领域插件。

> 案例回扣 讲义列出的反馈入口：空间/几何任务用生成器与约束检查；并发协议与故障诊断用运行、模型检查、故障注入；实验选择、记忆管理与需求澄清联系后续预测和任务结果。



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 5 · 原始讲义 · 相关讲解 00:09:16（不是同秒视频截图） · 原讲义第 5/39 页：增加可训练领域。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

00:18:34—00:20:12

回到软件开发：Intent—Specification—Implementation

课程把主题拉回软件工程：软件难，不只是代码难，而是人的意图（Intent）、可执行的规格（Specification）和最终实现（Implementation）之间始终存在间隙。版本控制只是管理变化的基础，真正的问题是怎样从空仓库构造出能工作的系统。

Review: 版本管理

软件是 Intent、Spec、Impl 的组合体

- 以文件系统快照的形式存储 (版本管理)
- 所谓 “软件开发”，就是不断推出新的版本，直到满足客户需求
  - 这其中很自然会遇到分叉、冲突……
  - 于是你能理解 SVN、Git、Jujutsu、……的设计原因

从零开始构造软件

- 上次起了个头：从空的项目 (模糊的 intent) 开始，我们可以做什么？

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 6 · 原始讲义 · 相关讲解 00:18:34（不是同秒视频截图） · 原讲义第 6/39 页：Review: 版本管理。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

教师给出的现代起点是 MVP。做得越少，错误和沉没成本越少；一个最小但接近最终产品的原型，可以尽快暴露“意图没想清、规格不合理、实现路径不通”等问题。AI 生成的首版代码可以被丢弃，它的主要价值是制造真实反馈。

因此，MVP 不是缩水成品，而是降低试错成本的认知工具。先用低成本实现去校准意图和规格，再逐步收敛到真正产品。

版本控制能够保存 Intent、Spec、Impl 的快照，组织分叉与合并，却不替我们决定第一版应当做什么。MVP 所创造的最重要产物可以是“原假设不对”的证据，而不一定是一份值得长期保留的代码。

整理者解释：把“最小”理解为聚焦高风险假设，而不是任意少做几个页面。一个只含漂亮界面、不能触达关键工作流程的原型，可能仍然无法检验真正的需求。

> 案例回扣 先生成可丢弃的 AI 原型，用它检验需求与架构，再决定正式实现。

讲义延伸 · 课堂未展开 · 原讲义第 38 页

LLM 把差距移到了默认决定中

MVP 能快速暴露问题，也会带入模型替我们补齐的选择。支付超时案例解释了：为什么代码跑通仍不代表意图已经对齐。

自然语言直接生成代码，省去了人工填写的部分中间步骤，却仍要为具体行为作选择。支付超时后等待、自动重试或直接失败，三种实现都可能跑通正常路径；重写一次代码，默认行为可能已经变了。一个并发实现还可能产生多条轨迹，不能只观察一条成功路径就宣称满足规约。维护“已决定、明确禁止、尚未决定”的边界，是生成式开发继续要承担的工作。



官方网页讲义课后补充｜截图汇编 · PDF物理页 38 · 原始讲义 · 官方讲义第 38/39 页：LLM 把差距移到了默认决定中。图文与本节问题相接；课堂未逐项展开，无对应口述时间。

00:20:12—00:25:23

Office Assistant：把设备抽象成接口

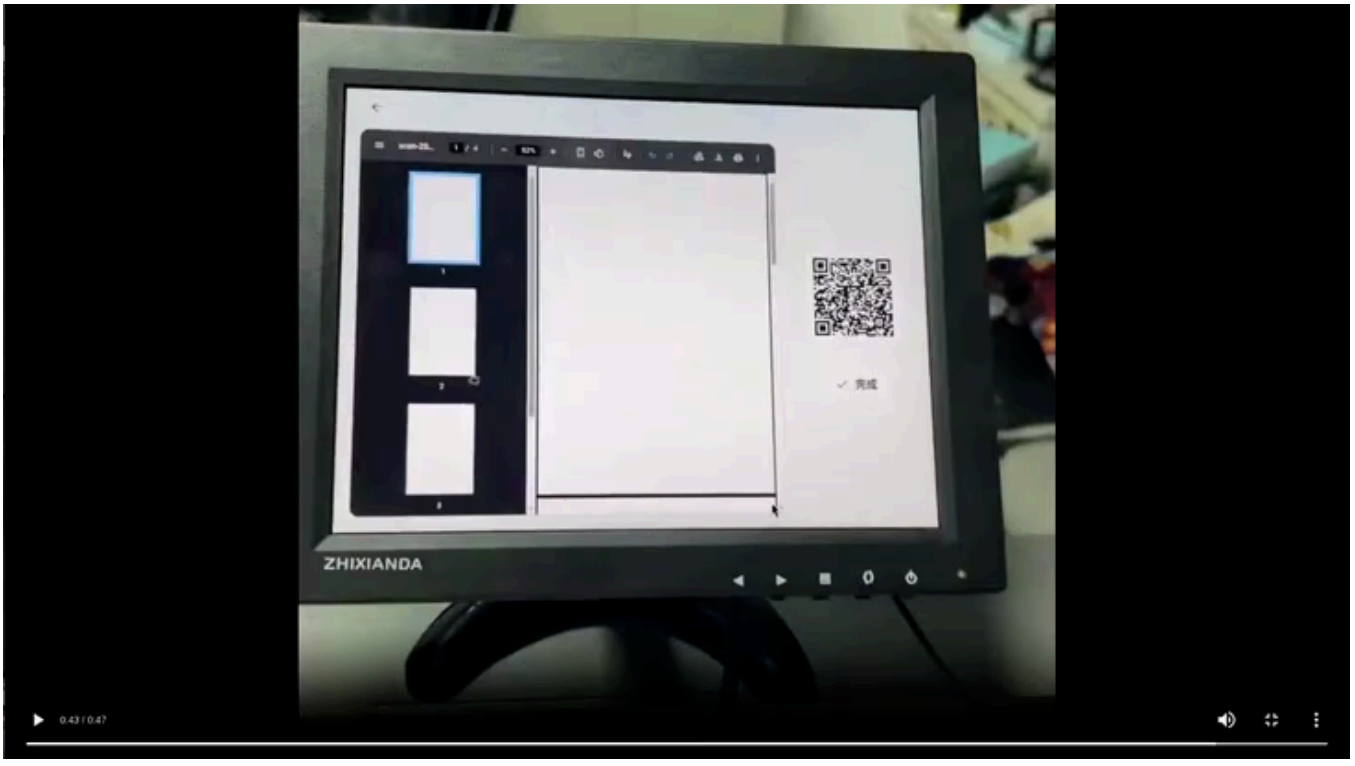
教师展示自己用 AI 构造办公室自助设备系统的过程。实验室已有扫描仪、标签打印机等设备，但厂商软件难用、平台割裂。解决办法不是继续适配每个 GUI，而是把设备能力抽象成稳定接口：扫描仪提供 scan（单双面、DPI、方向），打印机提供 print(image)。

一旦接口建立，硬件细节与上层产品被解耦。上层只面向“扫描”“打印”这些语义工作，底层可以替换具体设备和驱动。这是软件工程中典型的良好分解：用抽象缩小变

化传播的范围。

> 案例回扣 接口含义示例：scan 用方向、单双面与 DPI 描述扫描任务；print(image) 描述图像打印。这里是课堂的接口抽象，不是可直接调用的实际函数签名。

现场流程不仅是一张 UI：设备产生扫描结果，界面显示多页缩略图，并通过二维码提供文件获取入口。选用的 21:14.79 视频帧记录的就是这个完成状态；硬件连接、数据上传等过程细节来自相邻转写，不从截图中臆测。



视频原帧 00:21:14 · 21:14.79 实际视频帧：Office Assistant 演示中的扫描结果页、缩略图与二维码。保留真实完成状态，替代官方第7页的未加载视频占位；细小文字受源视频分辨率限制。

教师实际把打印机与扫描仪分给两个 agent 窗口处理。任务能够并行，依赖于他先划清设备边界；随后软件层通过统一接口组合两边产物。这也呼应了后文的组织、分工和依赖管理。

对于非标热转印打印机，AI 通过多轮试验识别通用驱动、调整深浅参数并打印带编号的测试标签，教师只需要给出“偏淡还是偏浓”的验收反馈。若再加入摄像头，整个校准闭环甚至可以自动完成。

参数校准不是“AI 凭空知道硬件”：模型先试用系统识别到的打印路径，再转向可用的通用方式；反复打带编号的标签，教师反馈浓淡。驱动名称在转写中不够清楚，未根据音似猜成某个标准协议。

> 案例回扣 标签打印机通过连续试打和人工反馈收敛热转印深度。

这个案例也说明 agent 工作方式：人给出可验收结果，agent 反复探测、执行、观察和修正；前端则按具体使用者的偏好定制，而不是为所有人设计一个笨重的通用软件。

> **整理者说明**
>
> 摄像头自动验收是教师提出的下一步设想；当前演示仍存在教师对打印浓淡的反馈。不能把“没有管理实现细节”改写成整个物理闭环完全无人参与。

> **整理者说明**
>
> “软件公司消亡”是教师用于强调成本结构变化的强判断，课堂后文也保留了软件工程智慧仍然有效这一限定。

00:25:23—00:30:49

Agent 时代：App 退化为可调用能力

教师把设备抽象继续推到移动应用：如果 agent 能实时理解意图并操作界面，那么“点外卖”“查数据”就是服务接口，App 的人机界面不再是唯一入口。长期看，服务可能更重视 API 和可调用能力，而不是让每家公司都维护复杂客户端。

他以豆包手机、平台服务整合和长尾硬件为例，讨论 agent 可能压低跨平台驱动、配套软件和外包开发的成本。过去一台小众打印机也要承担 Windows、macOS 驱动及 GUI 成本；未来用户可能直接让 AI 为自己的设备与流程生成定制软件。

课程预告下一次实验将把第一个 assistant 与真实物理设备连接，可能面对非标协议、难用 App、缺少 Linux 驱动等开放问题；这在当堂尚未正式布置。教师以自有电子墨

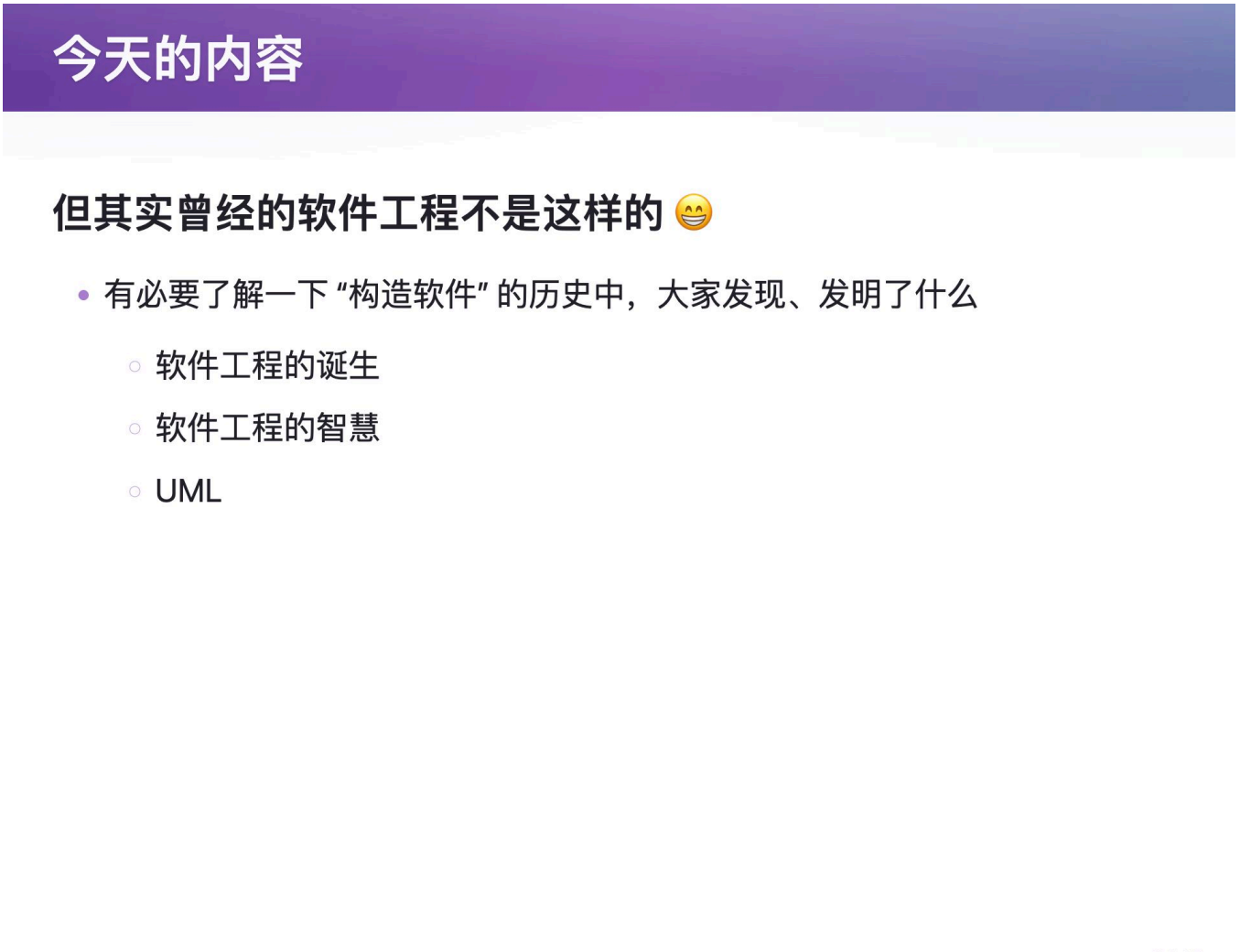
水屏为例，描述让模型分析 Android APK、理解蓝牙交互，再得到可编程接口的经历。

> 案例回扣 点外卖从“打开某个 App”变成 agent 调用一个服务能力。

> 案例回扣 逆向电子墨水屏 App 的蓝牙协议，绕开难用的人机界面。

但本课并未因此宣布软件工程无用。恰恰相反，AI 降低实现成本后，抽象、分解、反馈、可靠性等历史智慧仍决定人能否与 agent 合作构造好系统。

> **整理者说明**
>
> “App/软件公司消亡”是对软件成本与入口变化的强烈预测，不是已证实结论；接口的可靠性、权限、维护和业务责任没有因此自动消失。



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 8 · 原始讲义 · 相关讲解 00:25:23（不是同秒视频截图） · 原讲义第 8/39 页：今天的内容。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

00:30:51—00:39:14

1960s：昂贵计算机、年轻职业与巨大需求鸿沟

回到 1960 年代，硬件昂贵、工具原始，程序员必须从机器码和汇编层面工作，常借助流程图才能掌握控制流。程序员本身也昂贵且职业身份尚未稳定；Dijkstra 登记婚姻职业时，“程序员”甚至还不是被承认的职业。

软件还是新生事物的年代

回到 1960s：昂贵的码农

- 码农很贵：越早，编程工具越不成熟，码农越贵
  - 高智力需求的工种，成本极高
  - 当然那时候硬件更贵 😂
- 码农还经常缺少对客户的同理心、商业的理解等 “软技能”，也不懂业务

把客户的需求翻译成软件系统

- 软件企业的任务是把客户的需求翻译成软件系统
  - 客户不理解计算机能做什么
  - Intent-spec-impl gap 天然存在
- 这还是个年轻的职业：Dijkstra 回忆，1957 年结婚登记时，“程序员” 不被承认为职业，最后登记成了 “理论物理学家”。本人回忆

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 9 · 原始讲义 · 相关讲解 00:30:51（不是同秒视频截图） · 原讲义第 9/39 页：软件还是新生事物的年代。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

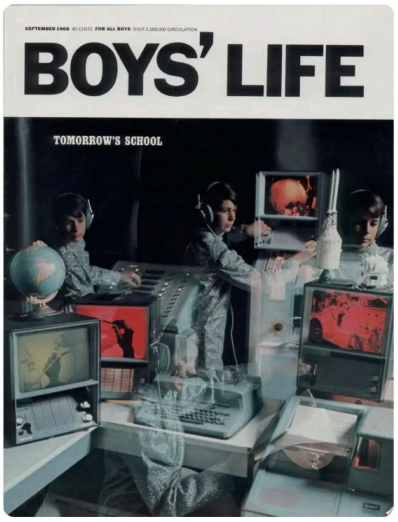
软件公司却必须把客户意图翻译成规格和实现。客户往往不知道计算机能做什么，技术人员又可能缺乏商业和用户同理心，双方之间天然存在巨大鸿沟。1968 年《Boys’ Life》描绘的“明日学校”已经想象实时判分、薄弱点辅导和个性化节奏，而工程现实仍是汇编、goto 和极其有限的算力。

早期客户对计算机的想象并不保守：讲义中的《Boys’ Life》已经把即时判分、薄弱环节辅导、按自身速度学习连在一起。难点在于“学会”怎么被测量，怎样从教育意图转成判分规则；同样的愿景可以对应很不相同的系统。

> 案例回扣 1968 年“明日学校”已经提出个性化学习愿景。

1968 年想象中的“明日学校”

计算机可以怎样教学生?



- 《Boys' Life》1968 年 9 月号：终端即时判分，针对薄弱环节辅导，让学生按自己的节奏学习。封面与原刊 · Computerized School House，第 24 页
- 课堂思考：“因材施教”是意图；程序怎样判断学生已经学会?

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 10 · 原始讲义 · 相关讲解 00:30:51（不是同秒视频截图） · 原讲义第 10/39 页：1968 年想象中的“明日学校”。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

软件工程与桥梁、材料等传统工程不同：物理工程受稳定自然规律和材料强度约束，需求常在成熟模式上增量演进；软件需求来自社会制度和人的偏好，可以高度定制、随时改变，并充满例外。

选课系统说明了这种复杂性：先修课究竟是硬限制还是软限制？教师能否特批？临近毕业是否必须开口子？很多需求只有进入真实制度和边界条件后才浮现。

选课案例还包括名额先到先得、抽签或专业保留，以及退课名额交给候补队首还是重新抢选。先修要求是硬规则还是允许特批，谁有权 override，都是系统需要执行的制度，不能由程序员碰巧采用的默认值决定。

> 案例回扣 选课系统的先修课规则需要处理专业保留、特批和毕业例外。

> **整理者说明**
>
> 课堂把传统工程与软件工程作比较，强调软件需求的社会性和可变性；这不表示桥梁等工程从无需求变化，也不表示软件不受物理资源约束。

软件工程和其他工程的区别

需求是高度定制化的

- 需求是**高度定制化**的
  - 也因此一直认为是高度需要人类智慧的 (编程，谁学谁知道)
- 以“选课系统”为例，同一个功能名称，可能对应完全不同的规则
  - 名额先到先得、抽签，还是按专业保留？
  - 不满足先修课要求时，老师能否特批？谁负责批准？
  - 退课空出的名额交给候补队首，还是重新开放抢课？

程序也在执行客户的制度与例外

- 这些选择决定了谁能选上课。我们的理解：业务规则与责任边界，本身就是要构造的软件的一部分。

11 / 39

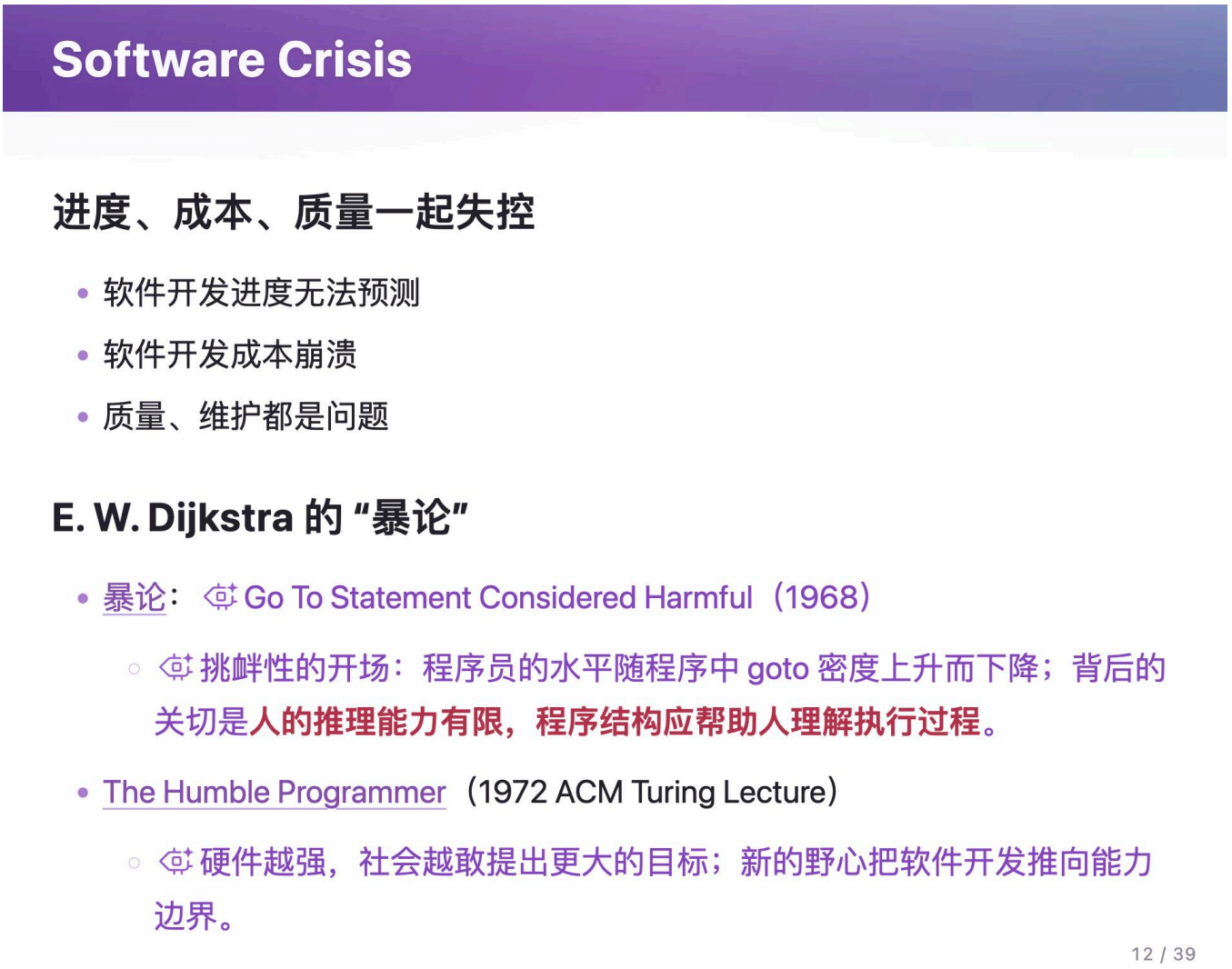
官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 11 · 原始讲义 · 相关讲解 00:30:51（不是同秒视频截图） · 原讲义第 11/39 页：软件工程和其他工程的区别。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

00:39:14—00:44:02

软件危机与“人能读懂的代码”

当软件开始承载更多现实意图，进度、成本和质量一起失控，软件危机出现。程序不再是一次性 Online Judge 小题：机器越强，系统承担的现实需求越多，不确定性和维

护成本也越高。



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 12 · 原始讲义 · 相关讲解 00:39:14（不是同秒视频截图） · 原讲义第 12/39 页：Software Crisis。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

Dijkstra 的 Go To Statement Considered Harmful 讨论控制结构如何帮助人理解执行过程。教师以 Linux 内核式的 goto release 为对照：出错时统一转到清理区，检查并释放已申请资源，可以是一种有纪律的 C 写法；它与任意跳转破坏控制结构是不同问题。

课堂给出的清理纪律是：新增资源申请时同步在统一出口增加对应检查和释放；中途失败不能绕过清理直接返回。真正被保护的是“所有路径都履行清理责任”的约束，goto 只是承载这个约束的一种机制。

> 案例回扣 C 程序用 goto release 汇聚资源释放，可以是受约束的正确机制。

《The Humble Programmer》进一步强调：面对不断增长的软件复杂度，程序员需要谦逊，寻找良好分解（nicely factored solutions），不要把个人聪明误当成可持续工程。

《The Humble Programmer》的课堂阅读还谈到硬件增强与软件危机的关系。机器更强之后，人会要求它承接更多现实目标，系统也更复杂；不能据此推导“买更快机器就能解决软件危机”。讲师回到 nicely factored solutions，把 Office Assistant 的设备接口分解作为当代例子。

这段历史给出一个判断：代码首先是长期协作与维护的媒介，其次才是让机器执行的指令。结构化、可读性和分解不是风格争论，而是降低系统性失败概率的工程手段。

> 案例回扣 一次性算法题调通后不再变化，因此不能代表真实软件维护。

00:44:02—00:50:12

1968 – 1969：软件工程命名与 Apollo 可靠性

1968 年 NATO 软件工程会议与 Margaret Hamilton 团队代表了学科成形：软件不再只是个人编程技巧，而是要在预算、时间、团队和硬件限制下可靠交付。

讲义补充：NATO 会议使用“Software Engineering”具有挑战传统观念的意味，要把理论基础和实践纪律带进软件生产。但会议也有人反对将一切编程统称为危机：成熟的排序、工资处理与前所未有规模的系统不是同一种困难。

“软件工程”成为共同议题

1968 年：一场刻意的“挑衅”

- 1968 年 10 月，50 多位专家在德国 Garmisch 参加 NATO 软件工程会议。会议报告，Background
- “Software Engineering”被刻意选作带挑衅意味的名称：软件生产也应有成熟工程学科的**理论基础与实践纪律**。报告前言

连“危机”也有争议

- 会上有人反对一概而论：排序、工资处理已有成熟办法；困难集中在规模和要求超出已有经验的系统。会议讨论，§7.1.2
- 由此理解“危机”：过去成功的经验，未必能支撑下一种规模与野心。

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 13 · 原始讲义 · 相关讲解 00:44:02（不是同秒视频截图） · 原讲义第 13/39 页：“软件工程”成为共同议题。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

Apollo 案例体现关键原则：“绝不可能发生的事，也可能发生。”软件必须假设硬件会故障、输入会违背预期、系统会在关键时刻报警，并设计优先级、恢复和降级机制。Apollo 11 下降阶段出现警报时，关键控制任务仍能继续，就是可靠性设计的价值。

Apollo 案例的细节不止是“出现警报仍能飞”：讲义记录 1201/1202 过载警报、恢复关键任务，以及事先测试恢复能力后地面团队才有依据继续任务。可靠性由系统机制、测试证据和操作判断共同构成。

> 案例回扣 Apollo 软件以优先级与恢复机制应对下降阶段警报。

把软件当作工程：Apollo

Margaret Hamilton 与她的团队

- Margaret Hamilton
  - 领导 MIT 仪器实验室的软件工程部门，参与 Apollo 飞行软件开发，推动软件工程观念获得承认。NASA / MIT 介绍

“能工作” 还包括发生意外的时候

- Apollo 11 登月下降时，计算机过载，出现 1201 / 1202 警报。亲历者解释
  - 软件的优先级与恢复机制，让关键的下降控制任务得以继续。
  - MIT 团队事先充分测试过恢复能力，地面团队才有依据判断可以继续登月。
- 这个案例里的工程工作包括：决定必须保住什么、实现恢复机制，并验证它确实有效。

14 / 39

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 14 · 原始讲义 · 相关讲解 00:44:02（不是同秒视频截图） · 原讲义第 14/39 页：把软件当作工程：Apollo。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

课程据此给出软件工程的工作定义：用系统化、可量化、有纪律的方法，把人的意图变成可工作、可维护、能随环境变化继续生存的软件，并覆盖开发、运行和维护的整个生命周期。

OJ 训练通常默认题目正确、条件固定且存在解。开放工程则可能遇到错误题意、改变的条件和不可靠硬件。因此，“正确”还包括怎样识别假设被破坏、怎样继续保住必要功能，而不是只求正常输入上的输出答案。

这也意味着从个人英雄主义转向组织能力。Checklist 不能保证成功，但能让一支普通团队重复执行关键动作，显著降低遗漏与失败概率。

> 案例回扣 把专家经验写成 checklist，使成功不再完全依赖某位英雄工程师。

软件工程的诞生

在成本、时间等约束下交付软件

- 在成本、时间等约束下，把人的意图变成可工作的软件系统。
  - 从大到团队组织管理，小到代码风格都会覆盖的学科

IEEE：把工程应用于软件

> IEEE: (1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software. (2) The study of approaches as in (1).

- 系统化、有纪律、可量化；覆盖开发、运行与维护。

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 15 · 原始讲义 · 相关讲解 00:44:02（不是同秒视频截图） · 原讲义第 15/39 页：软件工程的诞生。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

00:50:12—00:58:10

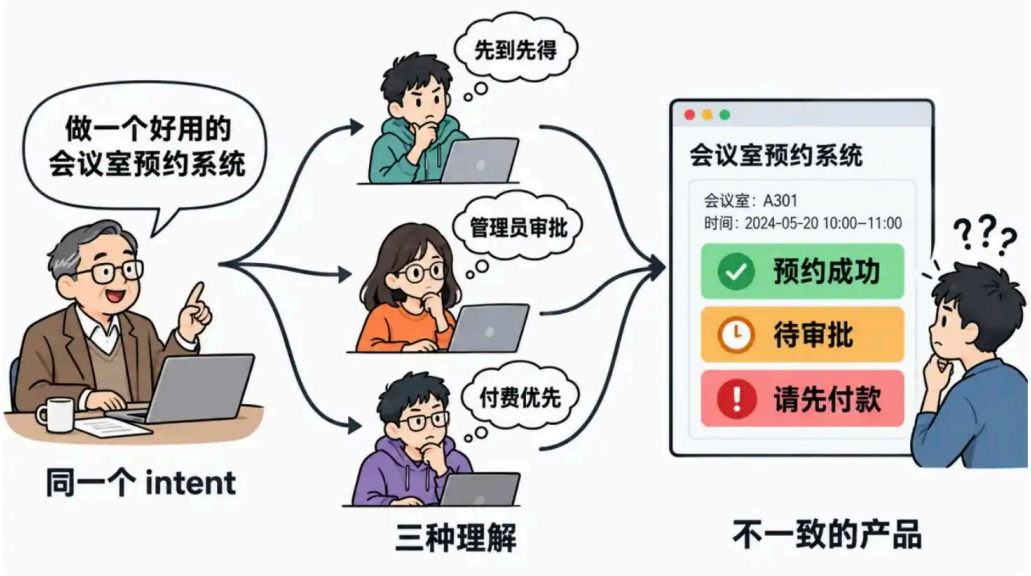
Royce 原文：顺序阶段为什么会诱发高成本返工

面对大型软件，Winston Royce 1970 年的经典论文讨论如何按时、按预算交付。教科书常把其中一张“需求→设计→编码→测试→运维”的图当成瀑布模型模板，但教师回到原文后指出：论文恰恰在说明这种单向流程为什么危险。

软件工程的智慧

回到 Pre-LLM 时代

- 接到开发任务，肯定不能立即安排码农干活——虽然最终产物是他们生产的
- 如何减少 **intent-spec-impl gap** 带来的影响?



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 16 · 原始讲义 · 相关讲解 00:50:12（不是同秒视频截图） · 原讲义第 16/39 页：软件工程的智慧。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

Royce 与瀑布模型

如何按时、不超预算地交付大型软件?

- Winston Royce: Managing the development of large software systems
  - 这篇论文画出了后来被称为 “瀑布模型” 的经典阶段图
  - 1970 年的背景：航天器任务规划、指令与飞行后分析，时间和存储都有硬约束。
  - 原文在这张图之后就指出：这样的实施方式风险很高，容易失败。

“瀑布假设”

- 希望迭代只发生在**相邻阶段**之间
- 实际测试中暴露的问题，却会一路回溯到**设计与需求**

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 17 · 原始讲义 · 相关讲解 00:50:12（不是同秒视频截图） · 原讲义第 17/39 页：Royce 与瀑布模型。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

小程序可由作者自己“分析—编码”两步完成；大型系统若把同样逻辑线性放大，测试就成为第一个真正遭遇物理事实的阶段。此前的需求与设计文档缺乏强约束，可能与代码脱节；等测试暴露意图、规格和实现漂移时，返工会沿依赖链回溯，造成超支和延期。

讲义把 Royce 的观察具体写成 timing / storage / I/O：测试时才第一次真正经历运行时间、存储占用、输入输出行为，前面只是分析。数学上算对的控制算法若赶不上控制周期，未必靠删几条指令能修复，可能要重做架构甚至改变需求承诺。

顺序模型有一个隐含乐观假设：返工主要只跨相邻阶段。真实测试若揭示根本性假设错误，会从实现一路退回设计、需求；此前所有依赖错误前提的成果都可能失效，成

本与进度风险因此放大。

> 案例回扣 测试是第一个让程序真正运行、让假设遭遇现实的阶段。

> 案例回扣 设计文档若随代码变化而不更新，会让上游错误拖到测试才爆炸。



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 19 · 原始讲义 · 相关讲解 00:50:12（不是同秒视频截图） · 原讲义第 19/39 页：Testing：第一次“被经历”。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

教师提醒，只背一张瀑布图会丢掉紧随其后的风险分析。课堂让 AI 解释这种误读如何流传；“图好画、好背、好出题”等是课堂对 AI 输出的讨论，不是本次已独立考证的教材传播史。

教师借阅读工具演示一种学习方法：用 AI 找到并读原始论文、保留页面和图片，再根据自己的问题做压缩。二手教材提供便利，却可能在多轮压缩中丢失作者真正论

证。

00:58:10—01:07:33

Do It Twice、TDD 与阅读原典：提前获得真实反馈

Royce 的补救核心是把问题发现时间提前，先用不完备但能暴露关键风险的 pilot model 运行一遍，再将结果反馈到设计。教师用今天的 MVP 帮助理解，但二者不是完全相同的概念：pilot 偏重技术和运行风险，MVP 通常还强调用户与业务假设。

“Do it twice”不是机械地完整开发两遍，而是承认第一次主要用来学习。试点失败的代价低，因此可以主动暴露假设、需求和架构问题。现代 TDD、原型和敏捷开发都能从这个原则获得解释。

> 案例回扣 试点程序先贯穿关键路径，再把经验用于正式系统。

Do It Twice

用今天的 MVP 理解 Royce 的 Pilot Model

- Minimum Viable Product：先做一遍，从中学习，再完善产品
- Royce 的 **do it twice**：先用 pilot model 经历关键问题
- Pilot 重点验证技术与运行风险；MVP 更关注用最小投入验证用户与业务假设。

用敏捷开发理解 “反复做 Pilot”

- 不必把 pilot 单独隔成一个大阶段，而是把试做、反馈、修正反复快速做
- 共同点：让关键假设尽早遇到证据；敏捷原则将频繁交付、合作与调整变成持续过程。

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 20 · 原始讲义 · 相关讲解 00:58:10（不是同秒视频截图） · 原讲义第 20/39 页：Do It Twice。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

TDD 把“提前遇到证据”具体化：先写输入、过程与预期输出，再写实现。错误的算法会在样例和测试阶段尽早暴露，而不是完成大量代码后才整体推倒。

TDD 要区分“先写期望”与“执行测试”：输入、边界情况和预期结果可以在实现前写出，真正运行并观察则依赖实现。先写测试是在明确需求，并不是魔法地在没有实现时获得全部运行证据。

> 案例回扣 先写测试用例，让错误设计在编码早期暴露。

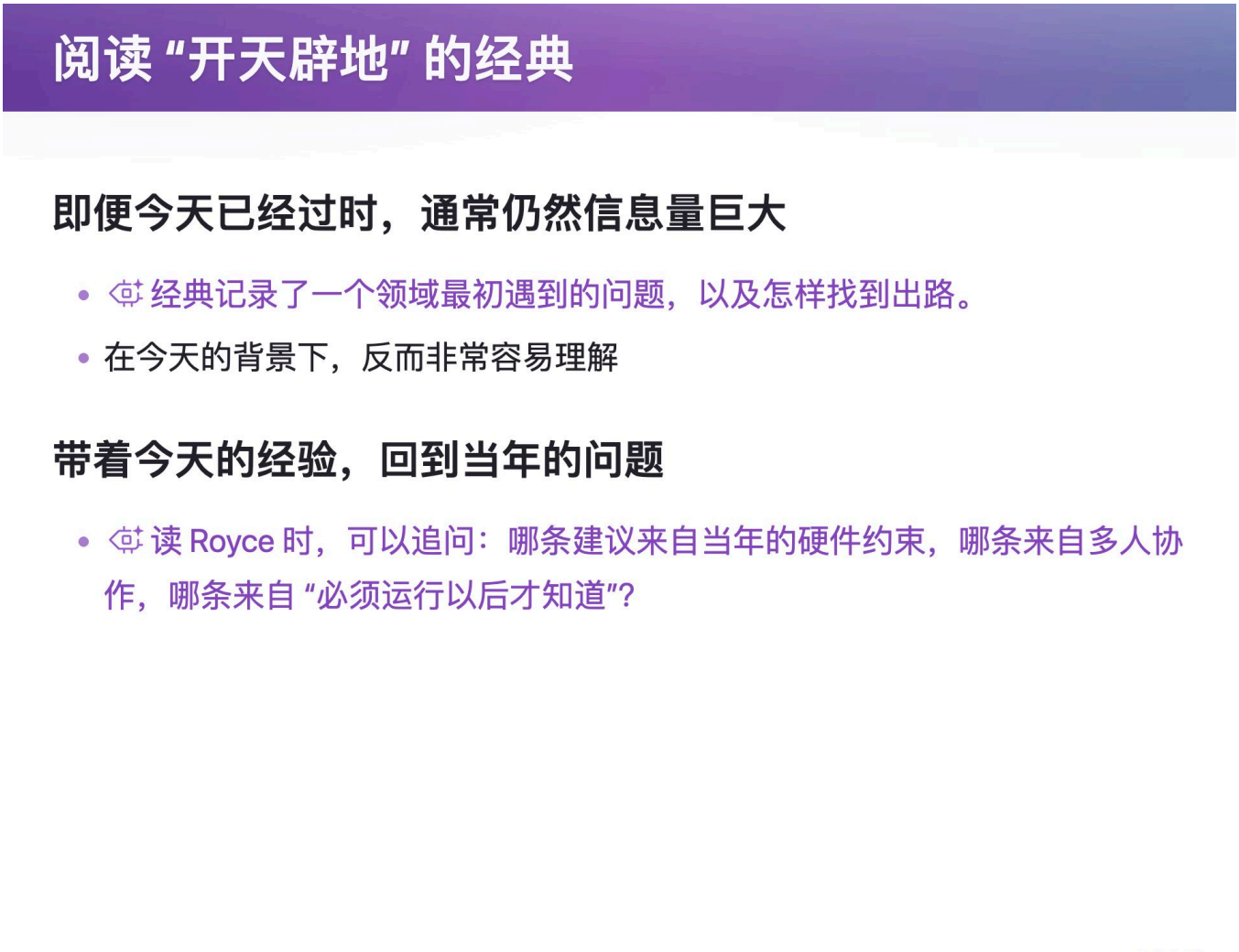
课程也花时间讨论知识压缩：过去查找一手资料成本极高，教科书是必要的有损压缩；AI 时代可以让每个人围绕自己的背景回到原文，形成个性化压缩。好的老师更像指出关键发现与原源，而不是替所有学生做同一种摘要。

课堂的原典工作流保留逐页文本和原图，可在命令行查阅、再让 AI 围绕疑问继续寻找原文。讲师认为应把知识拆成关键发现及其来源，让学生按自身背景重新压缩；这种方法仍需要检查 AI 是否准确代表原源。



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 18 · 原始讲义 · 相关讲解 00:58:10（不是同秒视频截图） · 原讲义第 18/39 页：AI 时代的一手信息。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

读经典要恢复作者面对的条件：哪些建议来自当时内存、时间与 I/O 的硬限制，哪些来自多人协作，哪些来自“不实际运行就不知道”。这样既能借用仍有效的原理，又不会把所有旧做法机械搬到今天。



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 23 · 原始讲义 · 相关讲解 00:58:10（不是同秒视频截图） · 原讲义第 23/39 页：阅读“开天辟地”的经典。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

> **整理者说明**
>
> 课程中关于教育投资被 AI “碾掉”的说法属于教师对产业变化的判断。

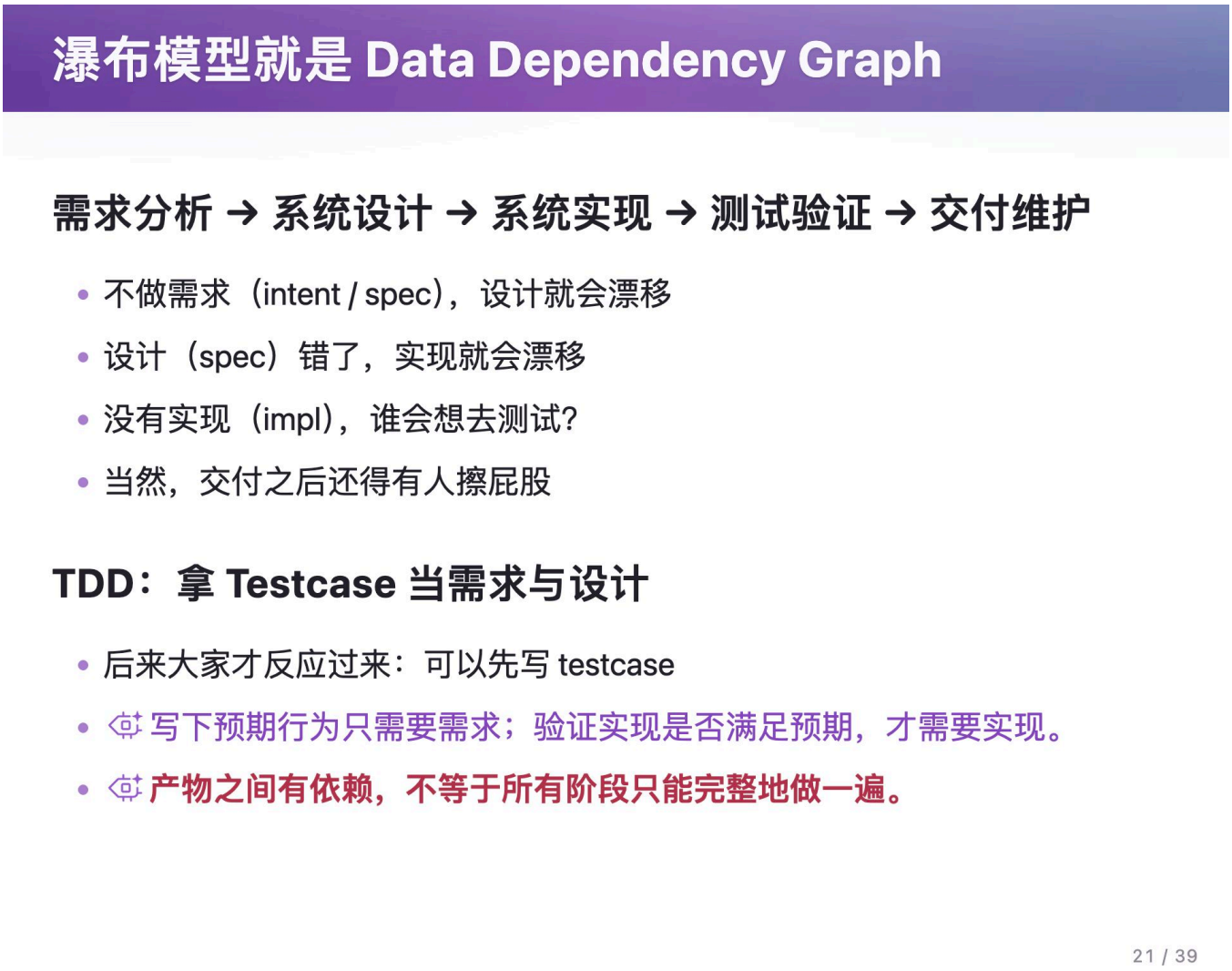
01:07:33—01:15:16

敏捷与数据依赖图：为何小步迭代有效

教师把敏捷解释为“do it infinite many times”：第一版甚至只要模块能互相打招呼、关键路径能跑通，然后持续演进。核心不是仪式，而是让关键假设尽早接触证据。

他用计算机科学语言重述瀑布：需求分析、设计、实现、测试、交付都是产生 artifact 的程序，artifact 之间形成有向无环的数据依赖图。某个上游需求或设计发生变化，所有下游依赖都可能失效，因此晚发现错误的成本很高。

> **整理者说明**
>
> “数据依赖图”是教师用于解释 Royce 模型的个人形式化，不是 Royce 论文原术语。

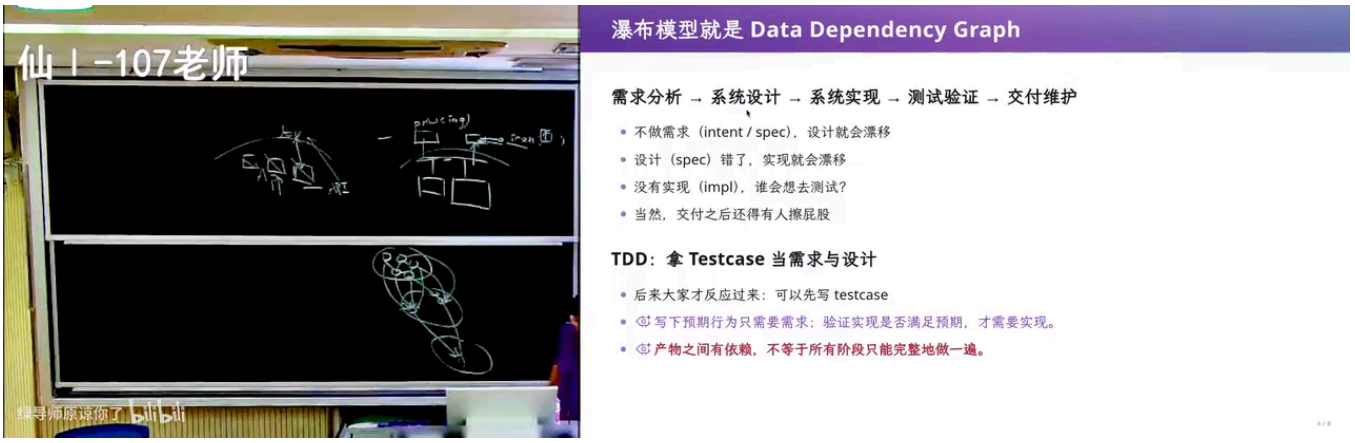


官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 21 · 原始讲义 · 相关讲解 01:07:33（不是同秒视频截图） · 原讲义第 21/39 页：瀑布模型就是 Data Dependency Graph。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

MVP 与敏捷的作用，就是控制依赖图的宽度和深度。不要一次性铺开巨大依赖，而是先走通优先级最高的窄切片，再逐步扩展；每轮都让意图、规格和实现重新对齐。

依赖图解释中的广度优先，是先铺开全部需求，再铺开全部设计，随后才把所有东西实现；深度优先式迭代则先沿一条重要路径走到可验证结果，再扩展相邻功能。这里

是教师的类比，不是要求开发工具实际运行 BFS/DFS 算法。



视频原帧 01:11:14 · 01:11:14.29 实际课堂全画幅：左侧保留教师画出的依赖节点与连线，右侧为数据依赖讲义。板书细字受原视频1280×410分辨率限制；依赖传播的完整文字解释见本节。

这个视角把 TDD、MVP、敏捷统一到同一原则：缩短反馈环、限制不确定性的传播范围、减少大规模返工。经典工作的价值，是在大家尚无答案时抓住了这些 first principles。

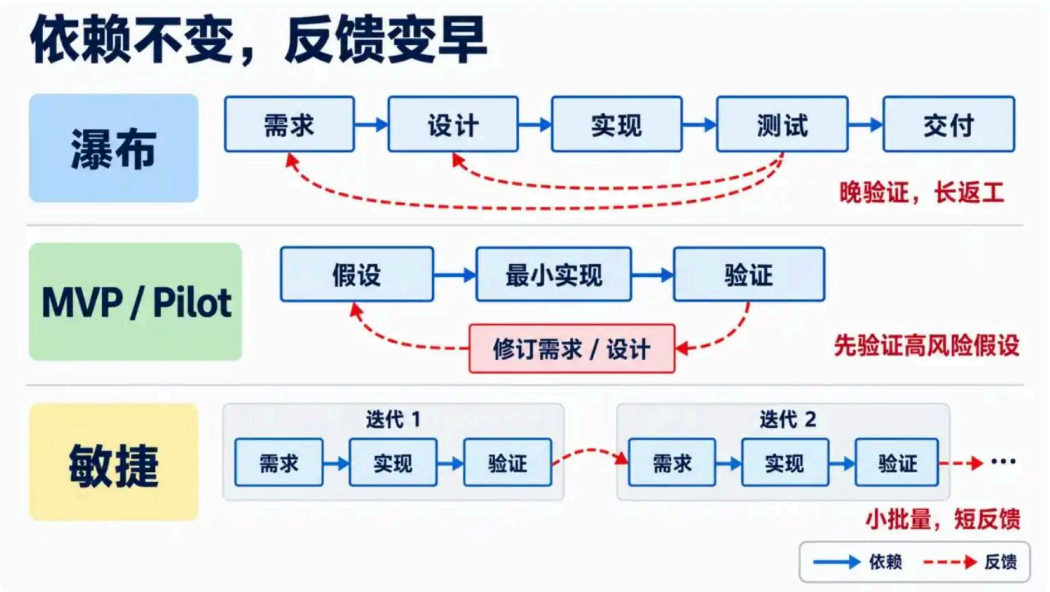
整理者解释：判断一个流程是否更好，应比较关键反馈来得多晚，以及反馈到来前已堆积多少依赖于该假设的工作。敏捷、MVP、TDD 都需要需求、设计、实现与验证，只是组织这些工作的粒度和反馈时点不同。

> 案例回扣 把第一版看作贯穿全栈的窄切片，而不是堆完某一层再进入下一层。

> 案例回扣 测试失败沿依赖边回溯，说明为什么上游错误越晚发现越昂贵。

用 Data Dependency 理解开发过程

MVP 与敏捷：让反馈更早回来



- 比较的不只是阶段顺序，还有：**多晚才得到反馈，得到反馈时已经做了多少依赖它的工作。**

22 / 39

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 22 · 原始讲义 · 相关讲解 01:07:33（不是同秒视频截图） · 原讲义第 22/39 页：用 Data Dependency 理解开发过程。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

01:15:16—01:23:14

软件工程的方法与工具：不是官僚主义，而是降风险

几十年发展后，软件工程为每个生命周期阶段创建方法与工具，目标是降低返工概率。需求阶段有访谈、评审和 checklist；设计阶段有模型与规范；编码后有静态分析、测试和坏味道检测。

学生常把长 checklist 看成繁文缛节，但其中一项“是否与客户最终确认”就能显著降低意图偏差。两份文档两两对照、每项打勾，看似低技术，实际把组织记忆和失败经验

固化成可重复流程。

Checklist 也可以看作半形式语言：每个检查项必须明确打勾或打叉，语义虽然由人执行，却比一篇随意散文更稳定。“文档两两对照”强制核查需求、设计和实现是否彼此矛盾；它背后是过去交付失败积累的组织经验。

> 案例回扣 客户确认 checklist 直接降低 Intent 漂移。

> 案例回扣 代码坏味道检测是生命周期中“编码阶段工具”的一个小分支。

软件工程的方法和工具

为每个阶段创建方法学和工具

- 软件工程：为每个阶段都创建**方法学**和**工具**，尽可能减少返工概率。

不同阶段，不同的 “语言”

- 在没有 LLM 这种 “万能编译器” 的时代，不同阶段需要不同的 “语言”：
  - 需求分析：文档、用例、界面原型 → “我们到底想要什么”
  - 系统设计：实体关系、状态迁移、模块和接口 → “系统怎样组织起来”
  - 代码实现
    - Computer Science 大部分课程集中在这里，算是 bottom-up 的学习方法了
- 在这里诞生了非常多有智慧的方法和工具。

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 24 · 原始讲义 · 相关讲解 01:15:16（不是同秒视频截图） · 原讲义第 24/39 页：软件工程的方法和工具。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

不同阶段需要不同“语言”。自然语言灵活但歧义大，编程语言精确但太靠近实现；软件工程因此不断发明位于二者之间的半形式语言，让需求、设计和代码之间更容易对

齐。

把自然语言需求变成设计文档，再变成代码，是逐步收紧允许行为的过程。课堂将 LLM 比作“万能编译器”，但同时坚持剩余 gap：模型能翻译已表达的要求，不能替产品负责人承担那些还没想清楚的业务选择。

AI 也可能改变教育顺序。过去必须先学代码，再理解产品和工程流程；当 AI 能提供资深编码能力时，初学者可以更早学习产品、需求和组织两千五百个并发“虚拟工程师”的管理心智。

教师用“可以先从产品开始做”讨论教学次序：初学者即使还不理解实现细节，也能先体验需求、产品与反馈过程，再逐步学会质量控制。这是教学主张，不意味着真实产品可以永久不理解或不检查代码。

01:23:14—01:28:00

测试、Property 与 Traceability：独立证据增加信心

软件工程不承诺“一次写对”，而是通过独立证据逐步增加信心。Property-based testing 让人描述系统性质，由工具生成大量输入寻找反例；普通测试则观察具体行为。每增加一层独立质量保障，对程序正确性的信心就多一分。

隐藏规格同样重要：程序不能崩溃、不能泄漏资源、必须保持数据结构不变量。若这些 property 能直接表达并由工具验证，就比仅靠程序员记忆更可靠；形式化证明则把信心进一步推高。

讲义的决策表案例是学生优惠遇到满减：究竟能否叠加，门槛按原价还是折后价计算？把条件组合逐一列出，会逼出遗漏；表格的价值不在形式漂亮，而在它要求每种组合都有明确结果。

QuickCheck 的完整排序例子包含两条性质：输出有序；输出保留输入每个元素及其出现次数。如果只写第一条，永远返回空数组也能通过。有性质、有测试不等于性质写对，测试本身仍要面对 Intent—Spec 的间隙。

> 案例回扣 由 property 自动生成输入，寻找破坏性质的反例。

方法和工具的智慧

需求与测试：把规则变得可检查

- **决策表**：把条件组合与预期结果对应起来，暴露遗漏和矛盾。ISTQB §4.2.3
  - 例：学生优惠遇到满减，究竟能叠加，还是只能选一个？
- **基于性质的测试**：QuickCheck 让人描述性质，由工具生成输入、寻找反例。
  - 例：排序结果既要有序，也要保留所有元素及其出现次数；只检查“有序”，返回空数组也能过关。

维护：改一句需求，会牵动什么？

- **需求追踪矩阵**：维护需求、设计与测试之间的关联，用于检查覆盖情况和分析变更影响。NASA
  - 例：“允许退课”改成“开课后一周内允许退课”，哪些设计和测试需要重新检查？

25 / 39

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 25 · 原始讲义 · 相关讲解 01:23:14（不是同秒视频截图） · 原讲义第 25/39 页：方法和工具的智慧。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

需求追踪矩阵（traceability）把文件系统里散落的需求文档、设计、源码和测试显式连成依赖图。每段代码能追到需求，每个需求能追到设计和测试，交叉检验就能减少遗漏和漂移。

退课规则从“允许退课”改成“只在开课后一周内允许”，会影响界面、权限检查、状态迁移和测试。追踪关系要支持从一个需求变化反查相关产物，也支持从代码或测试反

查其存在理由。

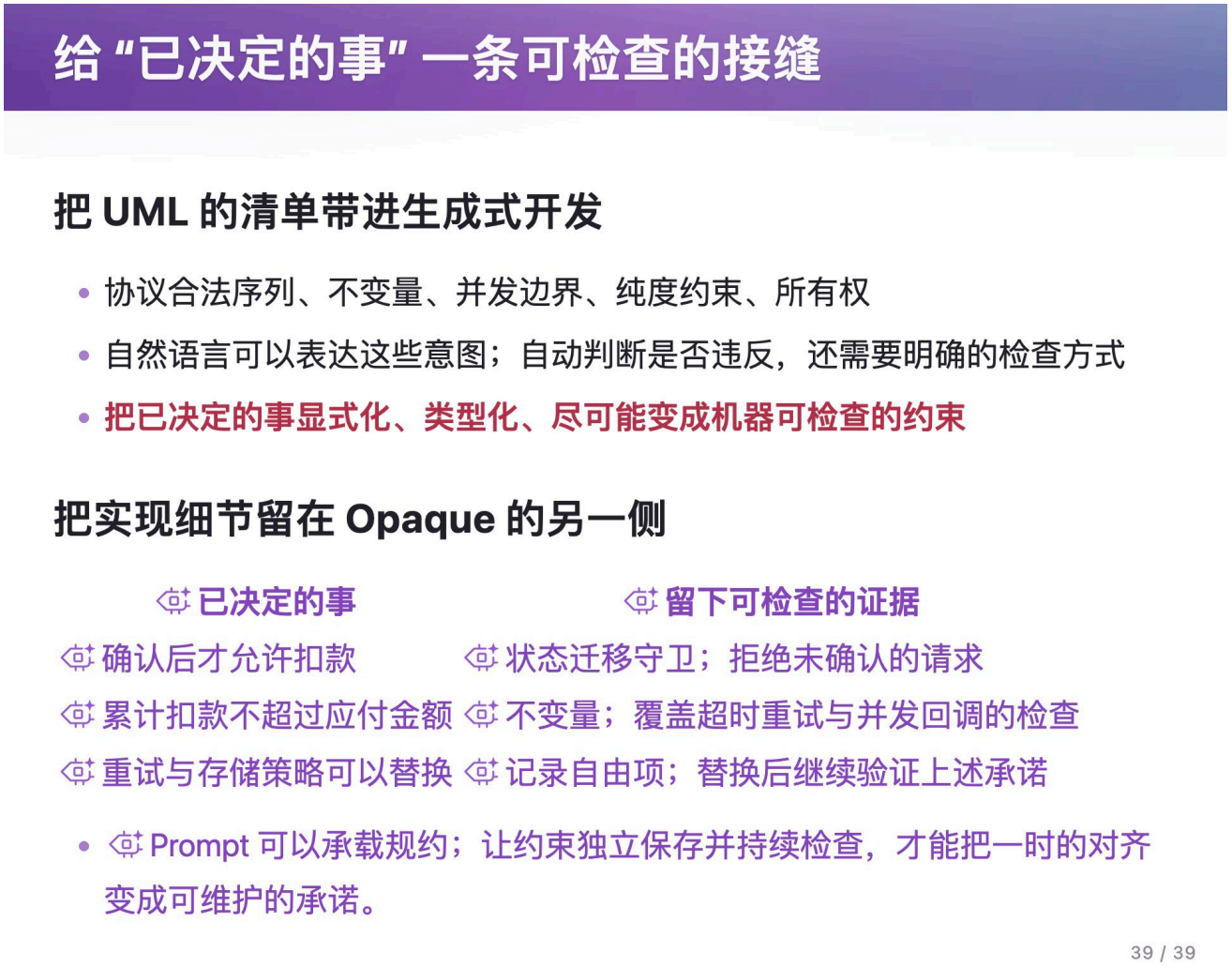
> 案例回扣 需求—设计—代码—测试建立双向链接，形成追踪矩阵。

讲义延伸 · 课堂未展开 · 原讲义第 39 页

给已决定的事情留下可检查证据

需求追踪把承诺连到产物；下面的支付清单进一步把每条承诺连到具体检查，使重写代码后仍能回查。

把业务承诺逐项接到检查方式：确认后才允许扣款，对应状态迁移守卫和拒绝未确认请求；累计扣款不超过应付金额，对应不变量以及超时重试、并发回调测试；重试与存储策略允许替换，对应明确记录的实现自由，替换后继续检查前两条承诺。协议序列、纯度、所有权和并发边界都可进入清单。Prompt 可以表达规约，但把约束独立保存并持续检查，才能让某次对话中的临时对齐成为可维护承诺。



官方网页讲义课后补充 | 截图汇编 · PDF物理页 39 · 原始讲义 · 官方讲义第 39/39 页：给已决定的事情留下可检查证据。图文与本节问题相接；课堂未逐项展开，无对应口述时间。

组织视角比“最强码农”视角更现实：大型项目不可能由一个人完成，管理者关心的是用多少人、多少预算，以何种流程稳定交付。AI 只是把可调度的工程能力放大，这一管理问题反而更突出。

01:28:00—01:32:34

Design by Contract 与可证明程序

Design by Contract 把规格写进程序结构。以 Eiffel 为例，一个函数包含说明、require 前置条件、do 实现和 ensure 后置条件；开发者甚至可以先不写实现，先明确模块承诺。

取款例子会逼迫设计者回答“余额不足怎么办”。可以设计一个要求调用方事先确认余额的底层接口，也可以设计一个接受余额不足并返回失败的业务接口；两种契约分配的责任不同。首个实验原型可能暂不覆盖某情况，但产品是否允许排除它必须被明确约定，不能默默让用户承担。

讲义给出的非透支账户契约：调用前 $0 < \text{amount} \leqslant \text{balance}$；正常返回后 $\text{balance} = \text{old(balance)} - \text{amount}$；对象对外可用时保持 $\text{balance} \geqslant 0$。若业务把余额不足视作正常结果，应另外规定失败返回且余额不变。

> 案例回扣 withdraw 的 require 表达余额条件，ensure 表达余额变化。

契约编程：Design by Contract

- Applying “design by contract” by Bertrand Meyer's Eiffel

调用者与实现者，各自负责什么?

- 像人类合作中的契约一样，接口写清双方的义务与权利。Eiffel 官方介绍
  - **前置条件**：调用者保证，调用前满足使用条件。
  - **后置条件**：在前置条件成立时，实现者保证正常返回后的结果。

例：不允许透支的账户取款

- 调用 `withdraw(amount)` 前：$0 < amount \leq balance$。
- 正常返回后：$balance = balance_{\text{old}} - amount$；$balance_{\text{old}}$ 是调用前的余额。
- **类不变量**：$balance \geq 0$，在对象对外可用时成立；各公开操作都须维护它。
- 这个设计也迫使我们做业务选择：如果 “余额不足” 是正常结果，就应把 “返回失败、余额不变” 写进接口约定。

26 / 39

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 26 · 原始讲义 · 相关讲解 01:28:00（不是同秒视频截图） · 原讲义第 26/39 页：契约编程：Design by Contract。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

讲义延伸 · 课堂未展开 · 原讲义第 33 页

契约与实现分层

取款例子划清了调用者与实现者的责任。把这个原则推广到查询、继承和协议，关键仍是将承诺独立于一次运行保存。

实现回答怎样做；契约回答在哪些前提下必须保证什么。前置条件、后置条件、不变量、协议状态机、isQuery 和多重性，各自承担一类要求。声明查询支付状态，不应顺手扣款；写下 isSubstitutable 是可替换性承诺，并不自动证明子类行为合格。实现跑通一次，只能说明那次执行满足观察到的条件。意图要独立于这次生成保留，并通过测试或验证检查。讲义也注明 UML § 6.3.3 没有把行为契约的责任全交给工具。

二、契约与实现分层

“必须恒真”与“碰巧成立”

- 实现告诉我们“怎样做”；契约声明“哪些条件下，必须保证什么”
  - 前置 / 后置条件、不变量、协议状态机、isQuery、多重性
- 前置条件不满足时行为“未定义”，是在划定契约边界
- 继承中的 isSubstitutable 表达可替换性承诺；重定义也必须遵守规则

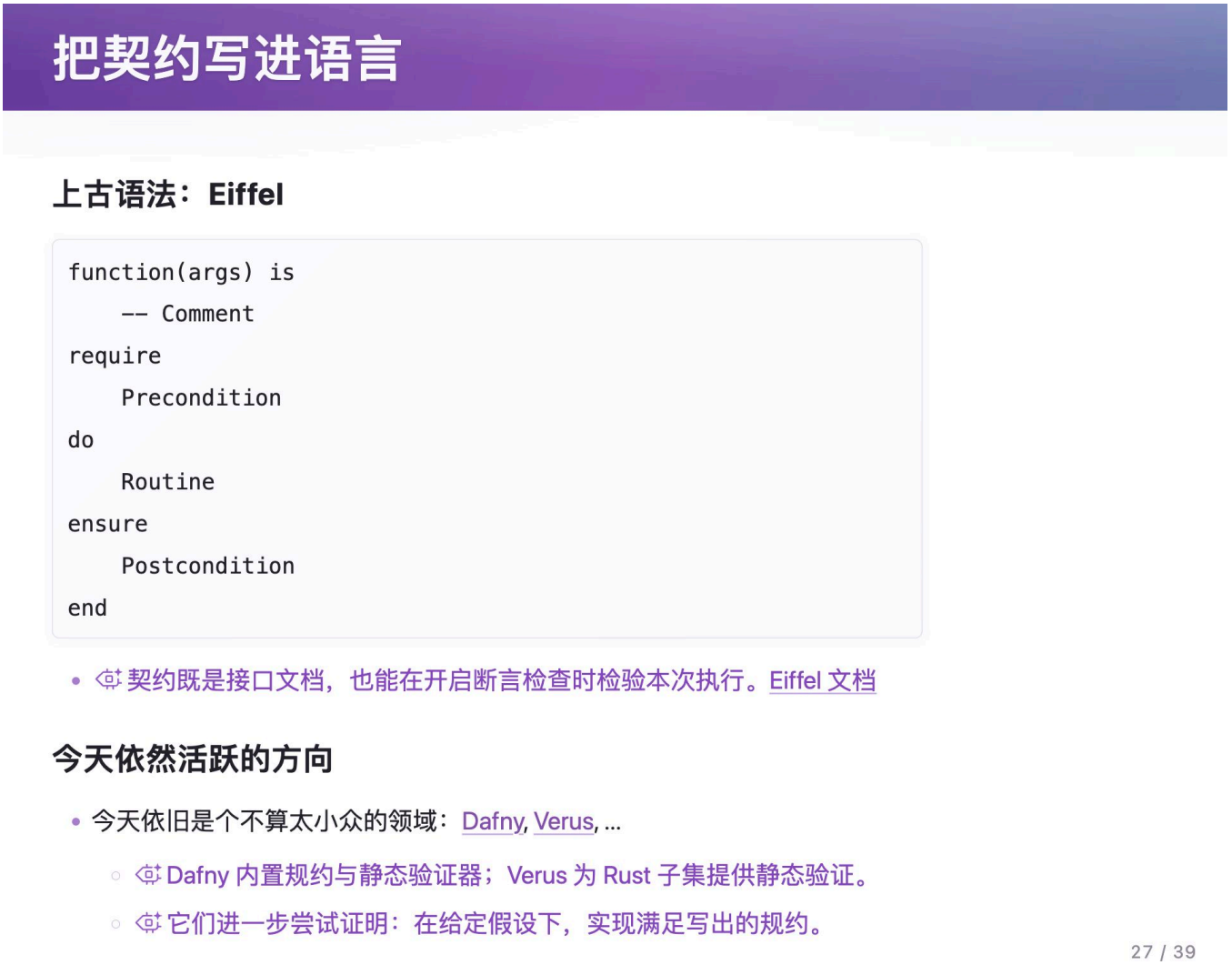
意图必须从这一次实现中独立出来

- 自然语言能声明“必须恒真”，但仅靠自然语言与样例，不能自动判定实现是否始终满足
- LLM 生成代码并跑通样例，只说明“这次跑通了”
- 例如：查询订单是否付费，不应顺手触发一次扣款；这才是 isQuery 要表达的承诺。
- isSubstitutable 本身不构成行为可替换性的证明；规范 §6.3.3 明确把满足行为契约的责任留给建模者。

33 / 39

官方网页讲义课后补充 | 截图汇编 · PDF物理页 33 · 原始讲义 · 官方讲义第 33/39 页：契约与实现分层。图文与本节问题相接；课堂未逐项展开，无对应口述时间。

Eiffel 风格结构依次为 function(args) is、说明、require 前置条件、do 实现、ensure 后置条件、end。先留下空 do、先写模块承诺，是把规格作为可以讨论的独立产物，而非代码完成后的补文档。



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 27 · 原始讲义 · 相关讲解 01:28:00（不是同秒视频截图） · 原讲义第 27/39 页：把契约写进语言。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

这一历史路线从自然语言需求、半形式模型、逻辑公式，一直走到编程语言和形式化验证。长期梦想是：开发者写下业务相关断言，编译器必须自动证明断言永远成立，否则程序无法编译。

教师认为大模型可能让这条路线加速。自然语言表达、契约生成、证明辅助与代码生成结合后，从意图到可验证实现的间隙可能显著缩小。

必须区分三层保证：运行时断言只检查本次实际执行；基于性质的测试通过样本搜索反例；静态验证在写明的假设及受支持逻辑内证明规约。教师对“未来任何业务断言若不能证明就拒绝编译”的设想属于愿景和时间预测，本课没有证明它对任意程序普遍可实现。

讲师以双向链表“向左再向右、向右再向左回到原结点”为不变量例子。证明这样的局部性质即使成功，也仍要检查它是否足以表达业务意图；正确地证明写错的规约，同样不能填平上游 gap。

> 案例回扣 链表“向左再向右回到自身”可写成结构不变量并尝试自动证明。

01:32:34—01:36:08

UML：在自然语言与代码之间对齐语义

UML 的野心是为面向对象世界提供统一建模语言。不同阶段原本使用不同语言：需求依赖自然语言，设计缺乏统一形式，实现使用 C、Java 等精确语言。UML 试图在中间层定义模型元素及其语义。

自然语言直接连接每个人不同的潜在空间，因此同一句“美女”会在不同大脑中生成不同形象。数学之所以容易对齐，是因为它从小规模公理逐步构造确定语义；UML 试图把能确定的设计关系从自然语言歧义中剥离出来。

UML 的统一对象不是所有自然语言含义，而是明确模型元素和各自语义。Student 类具有 name 字段是一种结构承诺；时序关系、状态或对象关系也可先独立于某次实现保存。未明确的部分仍需留白或继续澄清。

> 案例回扣 类图把 Student.name 表达成可映射到代码的结构。

Unified Modeling Language (UML)

能否有一种中间语言，统一整个面向对象世界?

- 人类的发展：自然语言（直接连接大脑 latent space）→ 数学语言（全世界自然对齐）
- UML 汇合了 Booch、OMT、OOSE 等方法：统一记号与语义，让团队和工具能使用共同的建模语言。OMG 历史说明

在没有代码时，尽量减少语义漂移

- 通过定义模型元素和（半形式化）**语义**，最大程度减少在没有代码时的语义漂移。
- 东西可能死了（LLM 时代谁还用 UML？），但设计时候的 first principles 一直指导我们。

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 28 · 原始讲义 · 相关讲解 01:32:34（不是同秒视频截图） · 原讲义第 28/39 页：Unified Modeling Language (UML)。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

讲义延伸 · 课堂未展开 · 原讲义第 31 页

决定与自由分开保存

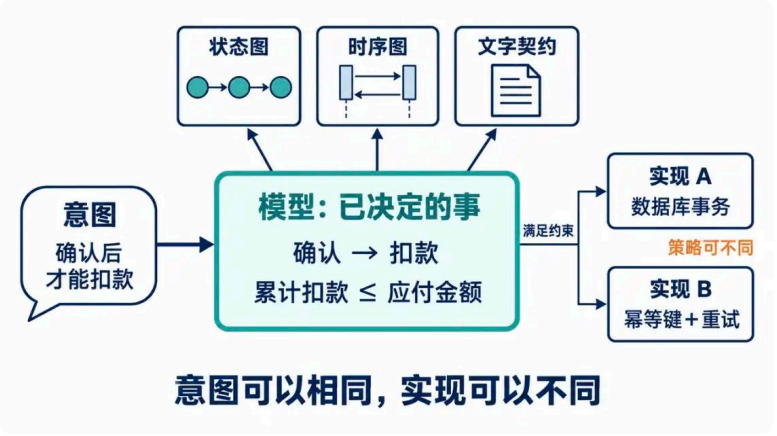
前面说模型能独立于代码保存承诺，支付案例进一步说明：究竟保存什么，允许哪些实现变化?

官方讲义第31页用支付展示“一个模型、多种实现”：意图是确认后才能扣款；模型固定确认先于扣款、累计扣款不超过应付金额；实现可以采用数据库事务，也可以采用幂等键与重试。选择不同技术路径，不应改变已承诺的业务边界。状态图、时序图和文字契约都是看同一组决定的窗口。一个好的模型会追问：用户连续点两次支付，到底算几次授权?

我对 UML 的理解

把决定存下来，把自由留出来

- 后面是 AI 的解读。我的判断：UML 的价值，是让**实现换了以后仍须遵守的决定**有一个可追问、可检查的家。



- 最有用的模型，会逼出原本没人问的问题：用户连续点了两次“支付”，究竟算几次授权？

31 / 39

官方网页讲义课后补充 | 截图汇编 · PDF物理页 31 · 原始讲义 · 官方讲义第 31/39 页：决定与自由分开保存。图文与本节问题相接；课堂未逐项展开，无对应口述时间。

讲义延伸 · 课堂未展开 · 原讲义第 32 页

图不等于模型

同一支付模型可以有状态图、时序图和文字契约；这就要区分模型本身与它的某一种画法。

模型保存元素、关系、身份和约束；图、文本和表格是它的不同视图。一张类图没画多重性，不能据此认定模型没有多重性约束。若多个视图各自改动而没有共同语义基础，需求、设计和测试会越来越不一致。讲义援引 UML 2.5.1 §2：规范自身的机器可读 XMI 定义与正文冲突时以 XMI 为准；这不等于任何项目都应无条件相信代码。UML DI 还为图的布局提供交换模型。只有截图会失去稳定的模型元素身份，使工具难以追踪“哪一个对象改变了”。

一、图 ≠ 模型

语义只有一个家，其余都是投影

- 图、文本、表格是底层模型的不同视图；一张图可以省略很多东西
  - **图上没画多重性，不得据此推断模型的多重性**
- 多视图各自演化，就会漂移；让意义收敛到共同的语义基座

凡需跨工具、跨时间存活的意图，必须落成显式数据

- UML 规范自身：所附机器可读 XMI 定义与正文冲突时，以机器可读定义为准。（UML 2.5.1，§2）
- 连图的布局也有模型：UML DI 让图的呈现信息能够交换
- 只交付截图，就失去了稳定的模型元素身份；改了一处，也难以知道哪些视图受影响。

官方网页讲义课后补充 | 截图汇编 · PDF物理页 32 · 原始讲义 · 官方讲义第 32/39 页：图不等于模型。图文与本节问题相接；课堂未逐项展开，无对应口述时间。

类图中的类、字段和关联可以映射到代码；时序图中的先后与并发关系也比自然语言叙述更明确。它不能完全消除歧义，却能减少从需求到设计再到实现的语义漂移。

讲义延伸 · 课堂未展开 · 原讲义第 35 页

把不同语义维度分开

类和关联被画出来之后，还要逐项确认每种关系的含义；一个箭头不足以替代这些决定。

当前值相同不等于对象身份相同；类型特化不等于集合成员限制；聚合、可导航性和关联端所有权也分别回答不同问题。学生对象能找到课程，不意味着删除学生要连课程一起删；关联端属性属于谁，也不直接等于运行时对象的生命周期归谁。一

个箭头若同时暗含可访问、归属、销毁责任，就很容易导致实现者各自猜测。模型应把这些决定分开表达，避免某个维度的变化偷偷改变其他承诺。

四、显式化与语义漂移

语义上独立的维度，必须有独立的表达

- 值 vs 身份；specialization 的内涵 vs subsetting 的外延
- 聚合 vs 导航 vs 所有权；同步 vs 异步；并发边界与责任归属
- 一个记号承载多个隐含含义，就是漂移的温床

一根线不能替你回答所有问题

- UML 把导航、聚合和关联端所有权分开；“可导航”不能直接当作“拥有”
- 学生对象能找到课程对象，不代表删除学生就该删除课程。
- “能访问”“谁定义这个属性”“生命周期是否绑定”，需要分别作出决定。
- UML 2.5.1，§11.5.4 区分这些维度；关联端的模型所有权，也不等同于运行时对象的生命周期。

35 / 39

官方网页讲义课后补充｜截图汇编 · PDF物理页 35 · 原始讲义 · 官方讲义第 35/39 页：把不同语义维度分开。图文与本节问题相接；课堂未逐项展开，无对应口述时间。

整理者校正：课堂把时序图的先后描述为便于对齐的例子，不能据此推断图上从上到下是全部事件的全局串行顺序。下面结合官方第 36 页区分 seq 弱序与 strict。

案例回扣 时序图用明确顺序减少并发描述的自然语言歧义。

讲义延伸 · 课堂未展开 · 原讲义第 36 页

顺序与并发的边界

顺序是上述语义边界的具体例子：图上排在前面，并不自动成为业务要求的先后关系。

UML 的 seq 是弱顺序：保留操作数内部的既有顺序，也保留同一生命线上跨操作数的顺序；不同生命线上的事件在不违反约束时仍可交错。图上短信通知画在仓库备货之上，不自动表示短信全部完成后才允许备货。如果业务确实要求严格先后，应增加明确依赖或使用相应严格顺序构造。“并发是默认”在这里用于提醒检查隐含顺序，不能替代各 UML 构造自己的语义。



从“并发是默认”理解欠规约

- 模型要陈述哪些顺序**必须成立**；不能把纸面位置当作全部执行顺序
- 具体到交互图，seq 是弱序：保留各操作数内部顺序，也保留同一生命线上跨操作数的先后。
  - 不同生命线上的事件，可以在不违反已有约束的前提下交错。

少画一条依赖，多留下哪些执行?

- 短信服务的通知写在上面，仓库服务的备货写在下面，不足以要求“通知结束后才能备货”。
- 确实需要这一承诺，就显式表达；strict 可约束片段之间的严格先后。
- “并发默认”是提醒我们检查隐含顺序；不同 UML 行为构造仍有各自的规则。

36 / 39

官方网页讲义课后补充 | 截图汇编 · PDF物理页 36 · 原始讲义 · 官方讲义第 36/39 页：顺序与并发的边界。图文与本节问题相接；课堂未逐项展开，无对应口述时间。

讲义延伸 · 课堂未展开 · 原讲义第 37 页

模型也要接受检查

语义写清楚以后，下一步是检查模型是否遵守这些规则，同时明确检查器到底保证到哪一层。

OCL（Object Constraint Language）可表达不变量和约束；Pin 的类型、多重性、PackageMerge 匹配规则、重定义合法性等都可能被检查。MOF 元模型组织模型元素及关系；Profile 可以增加领域约束，但不能违反扩展对象原本的 UML 语义。检查模型良构性与证明所有实现符合业务，是两件事。官方讲义还强调有些规则明确标注 Cannot be expressed in OCL，不能据此声称所有 UML 语义都已能机械判定。规则可以成为稳定检查的依据，但旧需求下的规则也可能过时。

五、语义即约束

让画图也能接受 “类型检查”

- 良构性不只靠散文，还靠 OCL 不变量等明确约束
  - Pin 的类型与多重性、PackageMerge 的匹配规则、重定义的合法性
- Profile 可以增加约束，不能违反它所扩展的 UML 语义
- UML 以 MOF 元模型组织自身：结构与约束也是可处理的数据

可机检的接缝，也有检查范围

- 检查 “这份模型写得合法”，与证明 “所有执行都遵守业务承诺”，是两项不同任务。
- 规范仍有 “Cannot be expressed in OCL” 的规则；并非全部语义都已形式化。§15.7.22 的 ObjectFlow 约束 就有实例。
- 约束是把惯例逐步收编为可机检规则的棘轮
  - 但需求改变时，约束也要改；检查器无法替你发现一条已经过时的承诺。

37 / 39

官方网页讲义课后补充 | 截图汇编 · PDF物理页 37 · 原始讲义 · 官方讲义第 37/39 页：模型也要接受检查。图文与本节问题相接；课堂未逐项展开，无对应口述时间。

教师判断传统 UML 在 AI 时代可能衰退，因为大模型成为自然语言与多种形式语言之间的“万能编译器”。但 UML 的 first principle 仍然有效：明确哪些意图必须定死、哪些可以留白，并让团队在关键语义上对齐。

> **整理者说明**
>
> “UML 已死”是教师对工具形态的判断；其对齐语义的目标在课程结尾被明确保留。

01:36:13—01:39:59

结尾：用 AI 重读 796 页 UML Spec

教师最后展示如何处理约 796 页的 UML 规范：先把文档转换成可检索、保留图片的材料，再按约 50 页分块并带少量重叠，让多个 agent 提取“仍然有效的软件工程智慧”，同时结合已有讲义生成课件。

讲师描述给多个 sub-agent 各分约 50 页、保留 overlap，要求提取软件工程智慧并结合课程已有材料备课。这是他展示的工作流，本笔记没有因此声称已复现他的整个 796 页规范阅读或其原始费用。

> **案例回扣** 长文档按 50 页分块、保留重叠，由多个 agent 提取核心工程思想。

课堂所说“¥699 套餐、18% 的五小时额度、折算约 ¥5.3”是讲师自己的成本折算。套餐额度比例不等于本次可核验的逐调用账单，也不代表读同一规范对每个人价格相同；本次处理没有可靠 token/费用数据，保留未知。

阅读 UML Spec

上大学的时候学过，感觉没学懂

- (怀疑老师可能也不太懂。)
  - 依稀有一些模糊的印象
  - PhD 时候也读了一些 papers
  - Kimi 送了 ¥699 的套餐，直接让 AI 备课，花去 18% 五小时额度 (～¥5.3)



官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 29 · 原始讲义 · 相关讲解 01:36:13（不是同秒视频截图） · 原讲义第 29/39 页：阅读 UML Spec。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

这个演示说明长文档阅读已可规模化，但也带来算力与教育公平问题。拥有更强模型和更多 token 的学生，学习效率可能显著更高；大项目维护会迅速消耗上下文，而针对单个知识点的学习可以利用更小的消化版本控制成本。

课程用一句话压缩 UML Spec：它从头到尾在回答“哪些 intention 必须定死，哪些可以留白”。半形式语言把可对齐的部分固定下来，把无法确定的部分显式保留。

> 案例回扣 796 页规范被压缩为“定死什么、留白什么”的对齐问题。

哪些意图必须显式钉死，哪些可以留白

用语言的方法解决 Intent-Spec-Impl Gap

> 796 页规范从头到尾只在反复回答一个问题：**哪些意图必须显式钉死，哪些可以留白**。它的全部设计都是对这个问题的回答。

和别人，或者下一时刻的自己对齐

- 如果你是从不 “朝令夕改” 的 deterministic people，其实你不需要用 UML
  - 因为你的行为总是可重复
- 但如果要和别人，或者下一时刻的自己对齐，UML 的 “语义对齐” 就很重要了
  - 稳定性解决自己会不会变；显式表达解决别人能不能知道。

官方网页讲义截图汇编（2026-10-02；非官方原生PDF） · PDF物理页 30 · 原始讲义 · 相关讲解 01:36:13（不是同秒视频截图） · 原讲义第 30/39 页：哪些意图必须显式钉死，哪些可以留白。已保留完整页面；时间仅关联本节主题，非同秒视频截图。

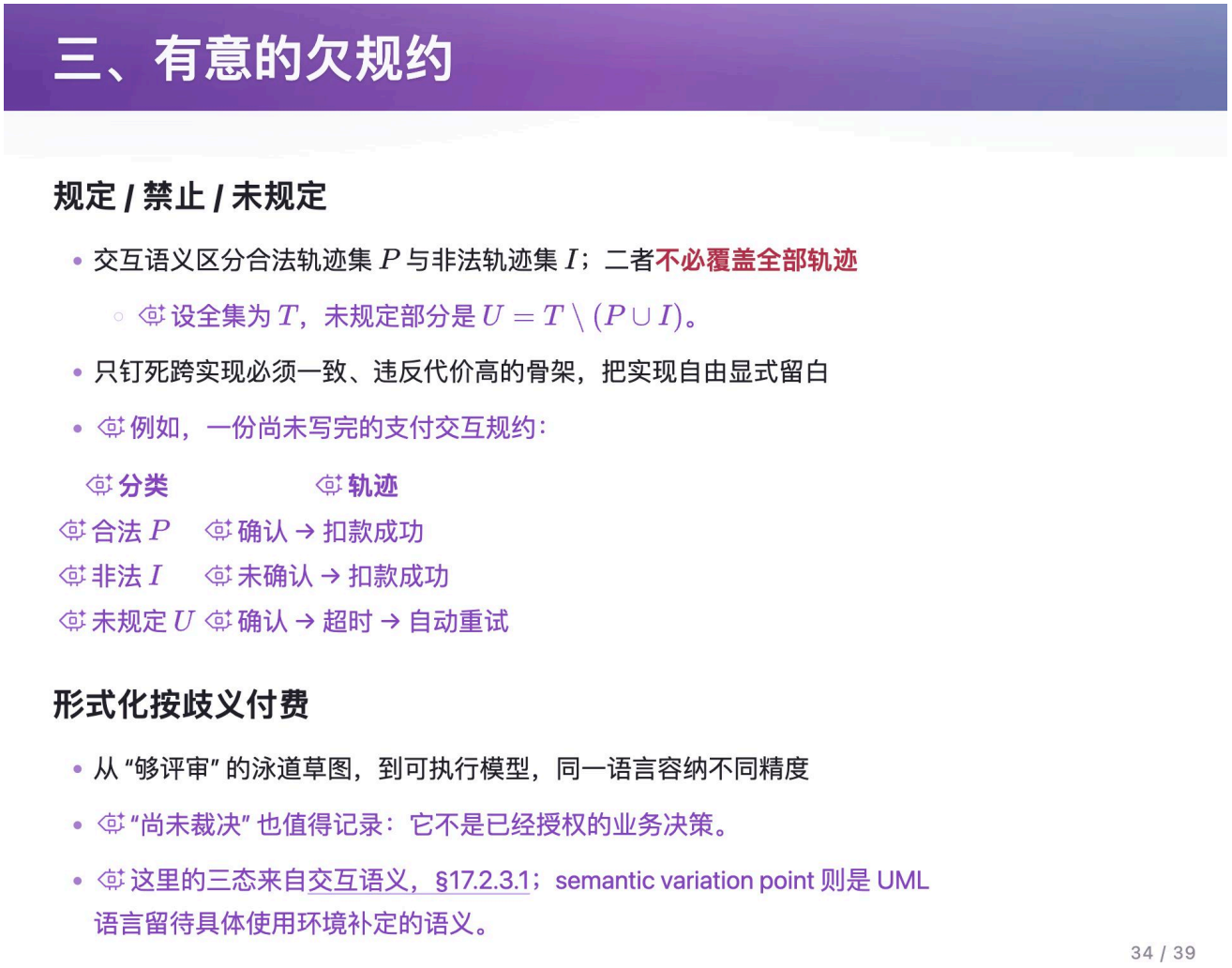
讲义延伸 · 课堂未展开 · 原讲义第 34 页

有意的欠规约

“哪些固定、哪些留白”还需要区分两种情况：允许实现者自由选择，或业务尚未作出决定。下面的三类轨迹把这个边界写清楚。

设全部执行轨迹为 T、合法轨迹为 P、非法轨迹为 I，那么尚未规定的部分为 U = T \ (P ∪ I)。合法与非法不必穷尽全部轨迹。支付例子：确认后成功扣款属于已规定；未经确认就扣款属于禁止；确认后请求超时是否自动重试可能尚未规定。“尚未规定”不表示业务已经批准任一选择。若要重试，仍需回答是否已扣款和如何避免重复扣款。用户模型留白与 UML 语言自身的 semantic variation point 是不同层

次：前者是该模型没决定，后者是语言保留给使用环境的语义选择。形式化应优先花在歧义大、代价高、需要机器继续推导的部分。



官方网页讲义课后补充｜截图汇编 · PDF物理页 34 · 原始讲义 · 官方讲义第 34/39 页：有意的欠规约。图文与本节问题相接；课堂未逐项展开，无对应口述时间。

最终落点仍是对齐：人与人、人与下一时刻的自己都不完全 deterministic。软件工程的历史，就是不断发明方法、语言和工具，缩小意图、规格、实现之间的漂移，并让变化和失败变得可管理。

最后约 01:39:50，讲师明确让学生自行阅读剩余 slides 并下课。这些材料现按问题融入相关章节，并就近标明"讲义延伸 · 课堂未展开"；它们补全课程资料，但不冒充本视频逐项讲授。

整理者说明

阅读记录：上一版记录了 20 个转写分块的全文阅读与关键结论回查。本次复用该记录，重点

> 复核图文落点、补充来源和自测交互，不把本次排版修改计作重新全量阅读。

自测：先回答，再核对

先用自己的话回答，再展开参考答案。答不清楚时，沿回查链接回到对应知识点。

问题 01

版本控制解决了什么，又没有替我们解决什么？

参考答案 · 核对思路

它保存并组织 Intent、Spec、Impl 的版本、分叉与合并；从空项目选什么需求、怎样得到第一份可验证实现，仍需要需求澄清和原型。

回查第 3 节 →

问题 02

Office Assistant 案例里的软件工程价值，为什么不只是“AI 会写驱动”？

参考答案 · 核对思路

把扫描和打印抽象成稳定能力接口，分离硬件细节与产品流程；把设备边界交给不同 agent，再用可验收结果和反馈收敛实现。

回查第 4 节 →

问题 03

“AI 校准打印机”是否已经是完全无人的闭环?

参考答案 · 核对思路

不是。课堂描述模型反复试打、教师反馈浓淡；摄像头自动判断是后续设想。

回查第 4 节 →

问题 04

为什么一条“支持选课”的需求远未完整?

参考答案 · 核对思路

还需规定名额分配、先修限制、特批权限、退课候补等制度与例外；程序最终一定会对这些情况产生某种行为。

回查第 6 节 →

问题 05

goto release 为什么不与结构化编程的目标必然矛盾?

参考答案 · 核对思路

受约束的统一清理出口能帮助人检查所有失败路径的资源释放；危险的是任意跳转导致控制结构无法推理。

回查第 7 节 →

问题 06

Royce 的问题为什么不能只靠在测试阶段多招人修 bug 解决?

参考答案 · 核对思路

运行约束可能暴露上游需求或架构错误；下游都依赖这些前提，局部改代码不够，需要提前用 pilot 接触风险。

回查第 9 节 →

问题 07

Pilot、MVP、敏捷有什么共同点和差别?

参考答案 · 核对思路

共同点是提前获取证据、限制未验证假设的影响。Pilot 偏技术运行风险，MVP 通常还验证用户业务假设；敏捷将短周期实现与反馈持续化。

回查第 10 节 →

问题 08

TDD 是否推翻“测试需要实现”这条依赖?

参考答案 · 核对思路

没有。写下期望行为可以在实现前进行；执行程序验证实际行为仍需要实现。

回查第 10 节 →

问题 09

为什么“排序输出有序”是一份不充分的规约?

参考答案 · 核对思路

永远返回空数组也有序；还需保留原输入元素及其出现次数。

回查第 13 节 →

问题 10

取款契约把余额不足放在前置条件外，会带来什么责任分配?

参考答案 · 核对思路

调用者需先保证余额足够。若真实产品必须接受不足余额请求并友好失败，应另写失败且余额不变的业务契约。

回查第 14 节 →

问题 11

一次断言检查通过，与静态验证有什么不同?

参考答案 · 核对思路

前者只检查一次执行；后者在给定假设与逻辑范围内证明规约。两者都不自动证明规约完整代表人的意图。

回查第 14 节 →

问题 12

UML 的“哪些固定、哪些留白”在生成式开发里对应什么?

参考答案 · 核对思路

独立保存必须守住的业务承诺、明确禁止的行为、尚未决定的部分；让实现自由存在，但每次重写仍检查承诺。

回查第 16 节 →